IACR Communications in Cryptology
ISSN 3006-5496, Vol. 2, No. 3, 30 pages.

https://doi.org/10.62056/ak5wom2hd

Check for updates

Towards Post-Quantum Bitcoin Blockchain using Dilithium Signature

Michel Seck[1] and Adeline Roux-Langlois[2]

[1] LTISI, CRISIN'2D, Ecole Polytechnique de Thies, Thies, Senegal
[2] Université Caen Normandie, ENSICAEN, CNRS, Normandie Univ, GREYC UMR6072, Caen, France

Abstract. Bitcoin is one of the famous cryptocurrencies in the world. It is a permissionless blockchain, and all transactions are stored in a public decentralized ledger. In its security design, Bitcoin utilizes various cryptographic primitives, such as hash functions and signature schemes. In the current version of Bitcoin, the Elliptic Curve Digital Signature Algorithm (ECDSA) is employed, which is not considered post-quantum secure due to the Shor's algorithm. Since December 2016, the National Institute of Standards and Technology (NIST) initiated a process to standardize certain post-quantum cryptographic primitives, including key encapsulation mechanisms (KEMs), public key encryption (PKE), and digital signature schemes. Dilithium, a lattice-based digital signature scheme, emerged as one of the winners of this competition and is recently standardized as ML-DSA (FIPS 204). In this work, we analyze the potential replacement of the ECDSA signature, the current signature in Bitcoin, with Dilithium, which is a post-quantum digital signature. This replacement will have a significant impact on many protocols within the Bitcoin ecosystem. The ECDSA algorithms are not only utilized for transaction signing and verification but also in wallet management. Bitcoin operates on a pseudonymous system rather than complete anonymity. To enhance privacy protection, the Bitcoin community has adopted a special type of (hierarchical) deterministic wallet as outlined in Bitcoin Improvement Proposal 32 (BIP32). We have constructed deterministic wallets by first designing DilithiumRK, a signature scheme with rerandomizable keys from Dilithium. Subsequently, we conducted a thorough security analysis and successful implementation of DilithiumRK.

Keywords: Signature scheme · signature with rerandomizable keys · deterministic wallets · hot/cold wallets · post-quantum cryptography · Dilithium · blockchain · Bitcoin

1 Introduction

Bitcoin [Nak08] is the first proof of concept (POC) of blockchain technology. It is currently the leading cryptocurrency adopted by millions of users worldwide, among thousands of existing cryptocurrencies. Bitcoin enables pseudonymous transactions, and there is no central authority to validate or control the system. Since its introduction in 2009 into the electronic monetary ecosystem, despite various attempts to breach the security of Bitcoin protocols, it has proven to be resilient. This is one of the reasons why it is widely embraced by millions of users around the world. In Bitcoin, as in numerous existing cryptocurrencies, digital signatures are used to authorize transactions and verify the ownership of bitcoins. To send or receive transactions on the Bitcoin network, a user must have addresses which are constructed from public keys and cryptographic hash functions. Each public key

E-mail: mseck@ept.edu.sn (Michel Seck), adeline.roux-langlois@cnrs.fr (Adeline Roux-Langlois)

This work is licensed under a "CC BY 4.0" license.
Received: 2025-04-04 Accepted: 2025-09-02

corresponds to an address, and vice versa. Bitcoin, like many cryptocurrencies, employs the ECDSA signature [Nat13], which is standardized [Nat13]. Some cryptocurrencies use the EdDSA (e.g. Ed25519) signature instead of ECDSA. Bitcoin is not an anonymous cryptocurrency and does not offer unlinkability or untraceability by design [KL18]. To enhance privacy protection, the Bitcoin community has proposed in BIP32 the use of deterministic wallets which are constructed from signature scheme with rerandomizable keys.

Signature scheme with rerandomizable keys. This is a signature scheme that supports additional properties, including rerandomization (blinding) for both public and private keys. It consists of five algorithms, the first three are standard signature algorithms: a key generation algorithm, a signing algorithm, and a verification algorithm. In addition to these three algorithms, there is an algorithm for updating a public key pk with a randomness μ and another algorithm for updating a private key sk with randomness μ. A derived session key pair (pk', sk') from a key pair (pk, sk) generated by the key generation algorithm should satisfy the standard correctness property of a signature scheme. A signature scheme with rerandomizable keys is highly useful in many security protocols that require privacy, such as Tor network, Bitcoin and other cryptocurrencies. Tor [Tor] is one of the most widely used anonymity tool. It hides the original IP address of a user connected to the Tor network. Like cryptocurrencies, some Tor network protocols use cryptographic primitives such as signature schemes with rerandomizable (bliding) keys [Tor, Hop13]. Signature schemes with rerandomizable keys were initially introduced by Fleischhacker et al. [FKM+16], who also demonstrated an instantiation from the Schnorr signature scheme. Since then, many constructions have been developed and utilized in the creation of deterministic wallets.

Deterministic wallets. For cryptocurrencies, a cryptographic wallet (or crypto wallet for short) is a tool that stores and manages cryptographic keys. In Bitcoin and similar cryptocurrencies, in order to ensure unlinkable transactions, a user, say Bob, generally generates multiple keypairs. Bob uses different keypairs for different transactions. To simplify the management and maintain a high level of security for these keys, deterministic wallets are proposed. In deterministic wallets [GS15, DEF+21, LYW+21], we have one master key pair (mpk, msk), and other key pairs are derived from this master key pair in a deterministic manner. Das et al. [DFL19] provided a formalization of such deterministic wallets in the hot/cold setting. In a hot/cold wallet setup, we have two wallets: a hot wallet that stores the public keys, which is permanently connected to the internet, and a cold wallet that stores the private keys, which can be never or rarely connected to the internet. The essential security properties for a deterministic wallet include unforgeability and unlinkability. Unlinkability in a wallet ensures that transactions sending coins to different session public keys (pk_i) derived from the same master public key (mpk) remain unlinkable. On the other hand, wallet unforgeability asserts that once coins are sent to the cold wallet, they must remain secure even if the hot wallet is compromise. Many deterministic wallets have been constructed in the last decade [DFL19, AADE+20, ER22, YLY+22, DEF+25, HLZ24, Hu23, SD23, DEMS24, SD25]. In the post-quantum setting, some isogeny-based deterministic wallets [SD23, DEMS24, SD25] have been proposed, as well as some lattice-based deterministic wallets [AADE+20, Hu23, HLZ24]. Alkadri et al.[AADE+20] constructed the first lattice-based deterministic wallet. They achieved this by designing the initial post-quantum secure signature scheme with re-randomizable public keys, built from generic lattice-based Fiat-Shamir signature schemes. They analyzed an instantiation of their generic wallet scheme using the qTESLA signature scheme[ABB+20]. Recently, Hu et al.[Hu23, HLZ24] proposed a lattice-based deterministic stateless wallet and demonstrated its security in the Quantum Random Oracle Model (QROM). They in-

stantiated their generic construction using Falcon[FHK+18], a post-quantum lattice-based signature scheme, and illustrated that their construction supports the cold/hot wallet paradigm. One of the drawbacks of current lattice-based deterministic wallets is the large size of public keys and/or signatures. For instance, at a security level of 160 bits, the total size of the public key and signature for Alkadri et al.[AADE+20] is approximately 42,159 bytes, whereas for Hu et al.[HLZ24], it is 24,652 bytes (see Table 2 in [HLZ24]).

BIP32 wallet. The BIP32 wallet [Wik18, GS15], introduced in 2012 within the Bitcoin system, has gained widespread adoption among Bitcoin users, making it one of the most frequently utilized wallet systems today. BIP32 is constructed thanks to the algebraic properties of the ECDSA signature. Although ECDSA is considered secure, there was initially no comprehensive security analysis, during its early years, of the deterministic BIP32 wallet, which became a standard in the Bitcoin community. Subsequently, authors such as Das et al. [DEF+21] and Groth and Shoup [GS22] conducted security analyses of signature schemes with rerandomizable keys derived from ECDSA. In 2021, Das et al. [DEF+21] performed a concrete security analysis of BIP32 wallets, providing the first evaluation of the protocol without any modifications. Their analysis revealed that BIP32 offers only 91 bits of security if they set the security parameter $\lambda = 128$ bits for ECDSA signature. They also proposed a modified scheme that increases security by 20 additional bits, resulting in a total of 111 bits of security, all without incurring additional costs. Additionally, they demonstrated how to design a generic hierarchical deterministic wallet based on a signature scheme with rerandomizable keys. Jens Groth and Victor Shoup's research [GS22] further explored the security aspects of ECDSA with additive key derivation, showing its relevance to BIP32 wallets.

Our Approach and Contributions. The deterministic wallet BIP-32 was adopted by the Bitcoin community as a standard without a comprehensive security analysis of the deterministic session keypair derivation from an ECDSA keypair. This oversight could have caused devastating consequences for the millions of Bitcoin and other cryptocurrency users. Given that the ECDSA signature is not post-quantum secure, it becomes evident that when quantum computers with sufficient computational power become available, the current signature scheme used in Bitcoin-like cryptocurrencies will need to be replaced by a post-quantum one. In this paper, we aim to address this security concern by conducting a thorough analysis of the security implications when replacing the ECDSA signature in Bitcoin-like cryptocurrencies with the new post-quantum signature scheme Dilithium, which is now standardized by NIST as ML-DSA (FIPS 204) [Nat24]. To construct Dilithium-based deterministic wallets, we have first designed DilithiumRK, a signature scheme with rerandomizable keys from Dilithium. We have shown that DilithiumRK satisfies the unlinkability of session public keys property which guarantees that an adversary who observes many session public keys cannot know with a non-negligible advantage if these keys are derived deterministically from the same master public key or not. We have also proved the unforgeability of DilithiumRK under honestly rerandomizable keys. We then derive deterministic wallets from DilithiumRK, which satisfy wallet unlinkability and wallet unforgeability properties.

Paper organization: This paper is organized as follows. The first Section 1 is devoted to the introduction. In Section 2, we present the foundational concepts and background information such as signature schemes with rerandomizable keys, and deterministic wallets. We have summarized the algorithms used to built the Dilithium signature scheme. In Section 3, we design DilithiumRK, a signature scheme with rerandomizable keys from Dilithium, and show how to obtain a (stateless or stateful) deterministic wallet from DilithiumRK. In Section 4, we conduct a security analysis of DilithiumRK and the

deterministic wallets derived from it. The unlinkability property of public keys as well as the unforgeability property of our wallet are proven. We show in Section 5 how to build a signature scheme with rerandomizable keys from the Dilithium signature with compression of public keys. We provide in Section 6 a comparison of DilithiumRK parameters and its variants with Dilithium levels 2 and 3 parameters, and also with other lattice-based deterministic wallets. The final section 7 is devoted to the conclusion and future work.

2 Preliminaries

In this section, we recapitulate various definitions of concepts relevant to our work such as signature schemes with rerandomizable keys, deterministic wallets, identification protocols and Dilithium functions.

2.1 Signature Scheme With Rerandomizable Keys

Definition 1 (signature scheme)**.** A signature scheme Σ is a triplet of algorithms $(\Sigma.\text{KGen}, \Sigma.\text{Sign}, \Sigma.\text{Verify})$ defined as follows.

- $\Sigma.\text{KGen}(1^\lambda) \rightarrow (pk, sk)$: is a randomized key generation algorithm which takes as input a security parameter λ and outputs a keypair (pk, sk) with pk a public verification key and sk a secret signing key.

- $\Sigma.\text{Sign}(m, sk) \rightarrow \sigma$: is a randomized (or deterministic) signing algorithm to sign a message m using sk, producing a signature σ of m.

- $\Sigma.\text{Verify}(m, \sigma, pk) \rightarrow b \in \{0, 1\}$: is a deterministic verification algorithm that takes as input a message m, a signature σ and a public key pk, and returns 1 (valid) or 0 (invalid).

A signature scheme $\Sigma = (\Sigma.\text{KGen}, \Sigma.\text{Sign}, \Sigma.\text{Verify})$ is $(1-\delta)$-correct if the following holds: For all $(pk, sk) \leftarrow \Sigma.\text{KGen}(1^\lambda)$ and all $m \in \{0,1\}^*$ we have

$$\Pr[\Sigma.\text{Verify}\,(m, \Sigma.\text{Sign}(m, sk), pk) = 1] \geq 1 - \delta.$$

When $\delta = 0$, we have a correct signature scheme. Note that in the following, if $\mathcal{A}$ is an adversary and $\mathcal{O}_1, \ldots, \mathcal{O}_s$ are oracles, then $\mathcal{A}^{\mathcal{O}_1,\ldots,\mathcal{O}_s}$ means that the adversary $\mathcal{A}$ has access to oracles $\mathcal{O}_1, \ldots, \mathcal{O}_s$. To consider a signature scheme as secure, it must satisfy the security notion of Unforgeability under Chosen Message Attacks **(Uf-CMA)**.

Definition 2 (Unforgeability under chosen message attacks of signature schemes [DFL19])**.** A signature scheme Σ is **Uf-CMA**-secure if for any PPT adversary $\mathcal{A}$,

$$Pr\left[\textbf{Uf-CMA}_\Sigma^\mathcal{A}(1^\lambda) = 1\right] \leq \varepsilon(\lambda)$$

where ε is a negligible function in the security parameter λ and the $\textbf{Uf-CMA}_\Sigma^\mathcal{A}$ is defined in the following game.

Game $\mathbf{Uf\text{-}CMA}^{\mathcal{A}}_{\Sigma}(1^{\lambda})$:

1: MList := $\emptyset$;
2: $(pk, sk) \leftarrow \Sigma.\mathrm{KGen}(1^{\lambda})$;
3: $(\sigma^{\star}, m^{\star}) \leftarrow \mathcal{A}^{\mathcal{O}\mathrm{Sign}}(pk)$;
4: $b_0 := m^{\star} \notin \mathrm{MList}$;
5: $b_1 := [\Sigma.\mathrm{Verify}(m^{\star}, \sigma^{\star}, pk) = 1]$;
6: **return** $b_0 \wedge b_1$

Oracle $\mathcal{O}\mathrm{Sign}(m)$:

1: $\sigma := \Sigma.\mathrm{Sign}(m, sk)$;
2: $\mathrm{MList} := \mathrm{MList} \bigcup \{m\}$;
3: **return** σ

Figure 1: The security game of unforgeability under adaptive chosen-message attacks **Uf-CMA** of a signature scheme $\Sigma = (\Sigma.\mathrm{KGen}, \Sigma.\mathrm{Sign}, \Sigma.\mathrm{Verify})$.

Definition 3 (signature scheme with rerandomizable keys)**.** A signature scheme with rerandomizable keys Σ_{RK} is a tuple of five algorithms where the algorithms ($\Sigma_{\mathrm{RK}}.\mathrm{KGen}, \Sigma_{\mathrm{RK}}.\mathrm{Sign}$, $\Sigma_{\mathrm{RK}}.\mathrm{Verify}$) are the standard algorithms of a signature scheme and the two others algorithms are defined over a randomness space $\mathcal{R}$ as follows.

- $\Sigma_{\mathrm{RK}}.\mathrm{UpdateSk}(sk, \mu_k) \rightarrow sk'$: is a deterministic algorithm to update a private key which accepts as input sk and an update $\mu_k \in \mathcal{R}$, and produces as output a rerandomized secret key sk'.

- $\Sigma_{\mathrm{RK}}.\mathrm{UpdatePk}(pk, \mu_k) \rightarrow pk'$: is a deterministic algorithm to update a public key which accepts as input pk and an update $\mu_k \in \mathcal{R}$, and produces as output a rerandomized public key pk'.

Definition 4 (Correctness)**.** A signature scheme with rerandomizable keys $\Sigma_{RK} = (\Sigma_{\mathrm{RK}}.\mathrm{KGen}, \Sigma_{\mathrm{RK}}.\mathrm{Sign}, \Sigma_{\mathrm{RK}}.\mathrm{Verify}, \Sigma_{\mathrm{RK}}.\mathrm{UpdateSk}, \Sigma_{\mathrm{RK}}.\mathrm{UpdatePk})$ is $(1-\delta)$-correct if the following holds:

1. For all $(pk_0, sk_0) \leftarrow \Sigma_{\mathrm{RK}}.\mathrm{KGen}(1^{\lambda})$ and all $m \in \{0,1\}^*$ we have that

$$\Pr[\Sigma_{\mathrm{RK}}.\mathrm{Verify}\,(m, \Sigma_{\mathrm{RK}}.\mathrm{Sign}(m, sk_0), pk_0) = 1] \geq 1 - \delta.$$

2. Moreover, after updating a keypair $(pk_0, sk_0) \leftarrow \Sigma_{\mathrm{RK}}.\mathrm{KGen}(1^{\lambda})$, $pk_i \leftarrow \Sigma_{\mathrm{RK}}.\mathrm{UpdatePk}(pk_0, \mu_i)$ and $sk_i \leftarrow \Sigma_{\mathrm{RK}}.\mathrm{UpdateSk}(sk_0, \mu_i)$ with $\mu_i \in \mathcal{R}$, we should have the following.

$$\Pr[\Sigma_{\mathrm{RK}}.\mathrm{Verify}\,(m, \Sigma_{\mathrm{RK}}.\mathrm{Sign}(m, sk_i), pk_i) = 1] \geq 1 - \delta.$$

A signature scheme with rerandomizable keys Σ_{RK} is correct if $\delta = 0$. One of the desirable security property for a signature scheme with rerandomizable keys is known as Unforgeability under Honestly Rerandomizable Keys (**Uf-CMA-HRK**). Note that the term honestly indicates that the randomness for the rerandomization is chosen uniformly at random from $\mathcal{R}$.

Definition 5 (Unforgeability under Honestly Rerandomizable Keys)**.** A signature scheme with rerandomizable keys Σ_{RK} is **Uf-CMA-HRK**-secure [DFL19] if for any PPT adversary $\mathcal{A}$, $Pr\left[\mathbf{Uf\text{-}CMA\text{-}HRK}^{\mathcal{A}}_{\Sigma_{RK}}(1^{\lambda}) = 1\right] \leq \varepsilon(\lambda)$, where the $\mathbf{Uf\text{-}CMA\text{-}HRK}^{\mathcal{A}}_{\Sigma_{RK}}$ is defined in the following game.

Game $\mathbf{Uf\text{-}CMA\text{-}HRK}^{\mathcal{A}}_{\Sigma_{RK}}(1^\lambda)$:

1: MList := $\emptyset$;
2: RList := $\{\epsilon\}$;
3: $(pk, sk) \leftarrow \Sigma_{RK}.\text{KGen}(1^\lambda)$;
4: $(\sigma^\star, m^\star, \rho^\star) \leftarrow \mathcal{A}^{\mathcal{O}\text{Rand},\mathcal{O}\text{RSign}}(pk)$;
5: $b_0 := m^\star \notin \text{MList}$;
6: $b_1 := \rho^\star \in \text{RList}$;
7: $pk^\star := \Sigma_{\text{RK}}.\text{UpdatePk}(pk, \rho^\star)$;
8: $b_2 := [\Sigma_{RK}.\text{Verify}(m^\star, \sigma^\star, pk^\star) = 1]$;
9: **return** $b_0 \wedge b_1 \wedge b_2$

Oracle $\mathcal{O}\text{RSign}(m, \rho)$:

1: **If** $\rho \notin \text{RList}$: **return** $\perp$;
2: $sk' := \Sigma_{\text{RK}}.\text{UpdateSk}(sk, \rho)$;
3: $\sigma := \Sigma_{RK}.\text{Sign}(m, sk')$;
4: MList := MList $\bigcup \{m\}$;
5: **return** σ

Oracle $\mathcal{O}\text{Rand}()$:

1: $\rho \leftarrow \mathcal{R}$; RList := RList $\bigcup \{\rho\}$;
2: **return** ρ

Figure 2: The security game of unforgeability under honestly rerandomizable keys $\mathbf{Uf-CMA-HRK}_{\Sigma_{RK}}$ of a signature scheme with rerandomizable keys

For application in deterministic wallets in the context of cold/hot wallet setting, it is sufficient to have a signature scheme with rerandomizable keys which satisfies the Unlinkability of session public keys **Unl-PK**.

Definition 6 (Unlinkability of session public keys)**.** A signature scheme with rerandomizable keys Σ_{RK} is **Unl-PK**-secure if for any PPT adversary $\mathcal{A}$, $Pr\left[\mathbf{Unl\text{-}PK}^{\mathcal{A}}_{\Sigma_{RK}}(1^\lambda) = 1\right] \leq \frac{1}{2} + \varepsilon(\lambda)$, where the $\mathbf{Unl\text{-}PK}^{\mathcal{A}}_{\Sigma_{RK}}$ is defined in the following game (adapted from [ELW23]).

Game $\mathbf{Unl\text{-}PK}^{\mathcal{A}}_{\Sigma_{RK}}(1^\lambda)$:

1: RPkList $\leftarrow \emptyset$;
2: $(pk, sk) \leftarrow \Sigma_{RK}.KGen(1^\lambda)$;
3: $s = 0$; $b \leftarrow \{0, 1\}$;
4: $b' \leftarrow \mathcal{A}^{\mathcal{O}\text{RandPk},\ \mathcal{O}\text{C},\ \mathcal{O}\text{RSign}}(pk)$;
5: **return** $b = b'$

Oracle $\mathcal{O}\text{RSign}(m, pk_\rho)$:

1: **if** $s = 1 \wedge pk_\rho = pk^\star_\rho$ **then**
2: **return** $\Sigma_{RK}.\text{Sign}(m, sk^\star_\rho)$;
3: **end if**
4: **If** $((pk_\rho, \rho) \notin \text{RPkList})$, **return** $\perp$;
5: **return** $\Sigma_{RK}.\text{Sign}(m, sk_\rho)$

Oracle $\mathcal{O}\text{C}()$:

1: **if** $s = 1$: **return** $\perp$;
2: $s = 1$;
3: **if** $b = 0$ **then**
4: $(\widehat{pk}, \widehat{sk}) \leftarrow \Sigma_{RK}.\text{KGen}(1^\lambda)$;
5: $sk^\star := \widehat{sk},\ pk^\star := \widehat{pk}$;
6: **else**
7: $sk^\star := sk,\ pk^\star := pk$;
8: **end if**
9: $\rho^\star \leftarrow \mathcal{R}$;
10: $pk^\star_\rho := \Sigma_{\text{RK}}.\text{UpdatePk}(pk^\star, \rho^\star)$;
11: RPkList := RPkList $\cup \{(pk^\star_\rho, \rho^\star)\}$
12: **return** $pk^\star_\rho$

Oracle $\mathcal{O}\text{RandPk}()$:

1: $\rho \leftarrow \mathcal{R}$; $pk_\rho := \Sigma_{RK}.\text{UpdatePk}(pk, \rho)$; RPkList := RPkList $\cup \{(pk_\rho, \rho)\}$
2: **return** pk_ρ

Figure 3: The security game of unlinkability of session public keys $\mathbf{Unl} - \mathbf{PK}_{\Sigma_{RK}}$ of a signature scheme with rerandomizable keys

The advantage of the adversary $\mathcal{A}$ is defined as follows:

$$\text{Adv}^{unl-pk}(\mathcal{A}) = \left| Pr\left[\mathbf{Unl\text{-}PK}^{\mathcal{A}}_{\Sigma_{RK}}(1^\lambda) = 1\right] - Pr\left[\mathbf{Unl\text{-}PK}^{\mathcal{A}}_{\Sigma_{RK}}(1^\lambda) = 0\right] \right|.$$

Definition 7 (Independent of session public keys, adapted from [ELW23])**.** A signature scheme with rerandomizable keys $\Sigma_{RK} = (\Sigma_{\text{RK}}.\text{KGen}, \Sigma_{\text{RK}}.\text{Sign}, \Sigma_{\text{RK}}.\text{Verify}, \Sigma_{\text{RK}}.\text{UpdateSk}, \Sigma_{\text{RK}}.\text{UpdatePk})$. Let $pk^0, pk^1, pk^2, \ldots, pk^n$ be $(n+1)$ public keys generated from $\Sigma_{\text{RK}}.\text{KGen}$. Sample uniform randomness $\rho_1, \rho_2, \ldots, \rho_n$. The **independent session public keys** advantage, denoted $\text{Adv}^{\text{ind}-\text{pk},n}_{\Sigma_{RK}}(\mathcal{A})$ is the advantage that an adversary has in distinguishing the following two distributions D_1 and D_2:

1. $D_1 : \Sigma_{\mathrm{RK}}.\mathrm{UpdatePk}(pk^0, \rho_1), \Sigma_{\mathrm{RK}}.\mathrm{UpdatePk}(pk^0, \rho_2), \ldots, \Sigma_{\mathrm{RK}}.\mathrm{UpdatePk}(pk^0, \rho_n)$
2. $D_2 : \Sigma_{\mathrm{RK}}.\mathrm{UpdatePk}(pk^1, \rho_1), \Sigma_{\mathrm{RK}}.\mathrm{UpdatePk}(pk^2, \rho_2), \ldots, \Sigma_{\mathrm{RK}}.\mathrm{UpdatePk}(pk^n, \rho_n)$

Definition 8 (Signing with oracle reprogramming [ESS21])**.** Let Σ_{RK} be a signature scheme (with rerandomizable keys) that relies on a random oracle H. We say that Σ_{RK} admits signing with oracle reprogramming if there exists a reprogrammed point extractor Ext and a forgery function **Forge** that takes in m, pk, and returns (y, σ) such that

$$\Sigma_{RK}.\mathrm{Verify}^{H:\mathrm{Ext}(m,\sigma,pk)\mapsto y}(m, \sigma, pk) = 1$$

where $H : x \mapsto y$ denotes the random oracle reprogrammed such that $H(x) = y$.

Definition 9 (Signature Distribution Change δ_Σ [ELW23, ESS21])**.** Let Σ_{RK} be a signature scheme (with rerandomizable keys) defined with respect to a random oracle H and a public key space $\mathcal{K}_{pk}$ that admits signing with oracle reprogramming via a point extractor Ext and a forgery function **Forge**. For a message and a public key m and pk, we consider the adversary's ability to distinguish the distribution of $(y_{forged}, \sigma_{forged}) \leftarrow \mathbf{Forge}(m, pk)$ from the distribution of $(y_{real}, \sigma_{real})$ where $\sigma_{real} = \Sigma_{RK}.\mathrm{Sign}(m, pk)$ and $y_{real} = H(\mathrm{Ext}(m, \sigma_{real}, pk))$ (i.e., the output of the hash on the input that would be reprogrammed).

The $L1$ distance between these distributions denoted as δ is defined as follows:

$$\delta = \sum_{\sigma,y} |Pr[\sigma_{real} = \sigma, y_{real} = y] - Pr[\sigma_{forged} = \sigma, y_{forged} = y]|$$

Lemma 1 ([ESS21])**.** *Let Σ_{RK} be a signature scheme (with rerandomizable keys) which admits signing with oracle reprogramming with $L1$ distance δ_Σ. Then There exists algorithms $\mathcal{B}$ and $\mathcal{C}$ such that*

$$\mathrm{Adv}^{unl-pk}_{\mathcal{A}} \leq \mathrm{Adv}^{\mathrm{ind-pk},n}_{\Sigma_{RK}}(\mathcal{B}) + q_H \mathrm{Adv}^{uf-cma-hrk}_{\Sigma_{RK}}(\mathcal{C}) + q_S \delta_\Sigma$$

2.2 Deterministic wallets

In the context of hot/cold wallets, we do not require the key generation algorithm to return a standard valid key pair. Instead, we have what is called a deterministic wallet scheme (both stateless and stateful), which can be seen as a relaxation of the signature scheme with rerandomizable keys. For a stateless deterministic wallet [HLZ24, Hu23], the key generation algorithm outputs a master public key mpk stored in the hot wallet, a master secret key msk stored in the cold wallet, and a static chaincode ch, which is a secret shared by both the hot and cold wallets. This type of wallet ensures the unlinkability of session public keys, unless the hot wallet is compromised by an adversary. In such a scenario, the adversary would gain access to the static secret chaincode and the IDs, allowing them to easily derive the session public keys. To address this weakness, stateful deterministic wallets [DFL19, AADE+20] were introduced. Instead of relying on a static chaincode, a state St is shared between both the cold and hot wallets. After each generation of a session public key using the state St, an ID, and the master public key mpk, the state is refreshed and the previous state is discarded. In this case, if an adversary compromises the hot wallet, he only gain knowledge of the IDs and the current state St_i. Then in this scenario the adversary cannot derive session public keys $pk_{ID_1}, pk_{ID_2}, \ldots, pk_{ID_{i-1}}$, where each session public key pk_{ID_j} is generated with state St_j for $j = 1, 2, \ldots i-1$. Thus, stateful deterministic wallets provide what is known as forward unlinkability, meaning that all session public keys derived before a state leakage remain unlinkable. The key ingredient used to construct deterministic wallets (both stateless and stateful) that ensure wallet

unlinkability and wallet unforgeability properties is a signature scheme with rerandomizable keys that satisfies the unlinkability of session public keys and the unforgeability properties. Consequently, our primary focus will be on developing such signature schemes and formally proving their security.

Definition 10 ([Hu23])**.** A stateless deterministic wallet scheme DW over a randomness $\mathcal{R}$ is a tuple of six algorithms (DW.KGen, DW.RandGen, DW.Sign, DW.Verify, DW.UpdateSk, DW.UpdatePk) where:

- DW.KGen$(1^\lambda) \to (mpk, msk)$: is a key generation algorithm which takes as input a security parameter λ and outputs a master keypair (mpk, msk) with mpk a public master key and msk a secret master key, as well as a chaincode ρ_{ID}.
- DW.RandGen$(ch, ID) \to \rho_{ID}$: The randomness generation algorithm takes as input a chaincode ch and an identity ID, then outputs a randomness ρ_{ID}.
- DW.UpdateSk$(msk, \rho_{ID}) \to sk_{ID}$: is an algorithm to update a private master key which accepts as input msk and an update $\rho_{ID} \in \mathcal{R}$, and produces as output a rerandomized secret key sk_{ID}.
- DW.UpdatePk$(mpk, \rho_{ID}) \to pk_{ID}$: is an algorithm to update a public master key which accepts as input mpk and an update ρ_{ID}, and produces as output a rerandomized public key pk_{ID}.
- DW.Sign$(m, sk_{ID}) \to \sigma$: is a signing algorithm to sign a message m using an updated secret key sk_{ID}, producing a signature σ of m.
- DW.Verify$(m, \sigma, pk_{ID}) \to b \in \{0, 1\}$: is a verification algorithm that takes as input a message m, a signature σ and an updated public key pk_{ID}, and returns 1 or 0.

Note that the public master key mpk and the secret master key msk can be seen as master seeds used to create valid keypairs (pk, sk). It means that we do not require the following property: for all $(mpk, msk) \leftarrow$ DW.KGen(1^λ) and all $m \in \{0, 1\}^*$; Pr[DW.Verify $(m,$ DW.Sign$(m, msk), mpk) = 1] \geq 1 - \delta$. The only required correctness property is the following.

Definition 11 (Correctness)**.** A stateless deterministic wallet scheme is $1 - \delta$-correct if after updating a master keypair $(mpk, msk) \leftarrow$ DW.KGen(1^λ),
$pk_i \leftarrow$ DW.UpdatePk(mpk, ρ_{ID_i}) and $sk_i \leftarrow$ DW.UpdateSk(msk, ρ_{ID_i}) we have the following.

$$\Pr[\text{DW.Verify}\,(m, \text{DW.Sign}(m, sk_i), pk_i) = 1] \geq 1 - \delta$$

A desired property of deterministic wallets is unlinkability which is explained in the following definition.

Definition 12 (Wallet unlinkability [HLZ24, Hu23])**.** The unlinkability for a stateless deterministic wallet guarantees that different session public keys that were derived from the same master public key should be unlinkable even if the adversary has access to a signing oracle with the session secret keys.

Formally, a deterministic wallet scheme DW : (DW.KGen, DW.Sign, DW.Verify, DW.RandGen, DW.UpdateSk, DW.UpdatePk) is **DWUNL**-secure if for any PPT adversary $\mathcal{A}$ the following holds:

$$Pr\left[\mathbf{DWUNL}^{\mathcal{A}}_{\mathrm{DW}}(1^\lambda) = 1\right] \leq \varepsilon(\lambda),$$

where ε is a negligible function in the security parameter λ and the $\mathbf{DWUNL}^{\mathcal{A}}_{\mathrm{DW}}$ is defined in the following game.

Game $\mathbf{DWUNL}_{\mathrm{DW}}^{\mathcal{A}=(\mathcal{A}_1,\mathcal{A}_2)}(1^\lambda)$:
1: RList := $\{\epsilon\}$;
2: $(mpk, msk, ch) \leftarrow \mathrm{DW.KGen}(1^\lambda)$;
3: $ID^\star \leftarrow \mathcal{A}_1^{\mathcal{O}\mathrm{RandPk},\mathcal{O}\mathrm{RSign}}(mpk)$;
4: **If** RList$[ID^\star] \neq \perp$: **return** 0;
5: $\rho_{ID}^0 := \mathrm{DW.RandGen}(ch, ID^\star)$;
6: $pk_{ID^\star}^0 := \mathrm{DW.UpdatePk}(mpk, \rho_{ID}^0)$;
7: $(\widehat{mpk}, \widehat{msk}, \widehat{ch}) \leftarrow \mathrm{DW.KGen}(1^\lambda)$;
8: $\rho_{ID^\star}^1 := \mathrm{DW.RandGen}(\widehat{ch}, ID^\star)$;
9: $pk_{ID^\star}^1 := \mathrm{DW.UpdatePk}(\widehat{mpk}, \rho_{ID^\star}^1)$;
10: $b \leftarrow \{0,1\}$;
11: RList$[ID^\star] := (pk_{ID^\star}^b, \rho_{ID^\star}^b)$;
12: $b' \leftarrow \mathcal{A}_2^{\mathcal{O}\mathrm{RandPk},\mathcal{O}\mathrm{RSign}}(mpk, pk_{ID^\star}^b)$;
13: **return** $b' = b$

Oracle $\mathcal{O}\mathrm{RSign}(m, ID)$:
1: **If** RList$[ID] = \perp$: **return** $\perp$;
2: RList$(ID^\star) := (pk_{ID}, \rho_{ID})$;
3: $sk_{ID} := \mathrm{DW.UpdateSk}(msk, \rho_{ID})$;
4: $\sigma := \mathrm{DW.Sign}(m, sk_{ID})$;
5: **return** σ

Oracle $\mathcal{O}\mathrm{RandPk}(ID)$:
1: **If** $pk_{ID} \in$ RList$[ID]$: **return**pk_{ID};
2: $\rho_{ID} := \mathrm{DW.RandGen}(ch, ID)$;
3: $pk_{ID} := \mathrm{DW.UpdatePk}(mpk, \rho_{ID})$;
4: RList$[ID] := (pk_{ID}, \rho_{ID})$;
5: **return** pk_{ID}

Figure 4: The security game of unlinkability $\mathbf{DWUNL}_{\mathrm{DW}}^{\mathcal{A}=(\mathcal{A}_1,\mathcal{A}_2)}$ of a deterministic wallet DW : (DW.KGen, DW.Sign, DW.Verify, DW.RandGen, DW.UpdateSk, DW.UpdatePk).

Definition 13 (Wallet Unforgeability [HLZ24, Hu23])**.** A stateless deterministic wallet scheme DW : (DW.KGen, DW.Sign, DW.Verify, DW.RandGen, DW.UpdateSk, DW.UpdatePk) is **DWUF**-secure if for any PPT adversary $\mathcal{A}$ the following holds:

$$Pr\left[\mathbf{DWUF}_{\mathrm{DW}}^{\mathcal{A}}(1^\lambda) = 1\right] \leq \varepsilon(\lambda)$$

where $\mathbf{DWUF}_{\mathrm{DW}}^{\mathcal{A}}$ is defined in the following game.

Game $\mathbf{DWUF}_{\mathrm{DW}}^{\mathcal{A}}(1^\lambda)$:
1: MList := $\emptyset$;
2: RList := $\{\epsilon\}$;
3: $(mpk, msk, ch) \leftarrow \mathrm{DW.KGen}(1^\lambda)$;
4: $(\sigma^\star, m^\star, ID^\star) \leftarrow \mathcal{A}^{\mathcal{O}\mathrm{RandPk},\mathcal{O}\mathrm{RSign}}(mpk, ch)$;
5: $b_0 := m^\star \notin \mathrm{MList}[ID^\star]$;
6: $b_1 := [\mathrm{RList}[ID^\star] \neq \perp]$;
7: $(pk_{ID}^\star, \rho_{ID}^\star) := \mathrm{RList}[ID^\star]$;
8: $b_2 := [\mathrm{DW.Verify}(m^\star, \sigma^\star, pk_{ID}^\star) = 1]$;
9: **return** $b_0 \wedge b_1 \wedge b_2$

Oracle $\mathcal{O}\mathrm{RSign}(m, ID)$:
1: **If** RList$[ID] = \perp$: **return** $\perp$;
2: RList$(ID^\star) := (pk_{ID}, \rho_{ID})$;
3: $sk_{ID} := \mathrm{DW.UpdateSk}(msk, \rho_{ID})$;
4: $\sigma := \mathrm{DW.Sign}(m, sk_{ID})$;
5: MList$[ID] := \{m\}$;
6: **return** σ

Oracle $\mathcal{O}\mathrm{RandPk}(ID)$:
1: $\rho_{ID} := \mathrm{DW.RandGen}(ch, ID)$;
2: $pk_{ID} := \mathrm{DW.UpdatePk}(mpk, \rho_{ID})$;
3: RList$[ID] := (pk_{ID}, \rho_{ID})$;
4: **return** pk_{ID}

Figure 5: The security game of unforgeability $\mathbf{DWUF}_{\mathrm{DW}}^{\mathcal{A}}$ of a deterministic wallet DW : (DW.KGen, DW.Sign, DW.Verify, DW.RandGen, DW.UpdateSk, DW.UpdatePk).

Definition 14 ([DFL19, AADE+20])**.** A stateful deterministic wallet scheme SFW is a tuple of five algorithms (SFW.KGen, SFW.Sign, SFW.Verify, SFW.UpdateSk, SFW.UpdatePk) where:

- SFW.KGen$(1^\lambda) \rightarrow (mpk, msk; St_0)$: is a key generation algorithm which takes as input a security parameter λ and outputs a master public key mpk, a master secret key msk as well as an initial state St_0.
- SFW.UpdateSk$(msk, ID, St) \rightarrow (sk_{ID}, St')$: is an algorithm to update a private master key which accepts as input msk, an identity ID and a state St, and produces as output a rerandomized secret key sk_{ID} and a new state St'.

- SFW.UpdatePk$(mpk, ID, St) \rightarrow (pk_{ID}, St')$: is an algorithm to update a public master key which accepts as input mpk, an identity ID and a state St, and produces as output a rerandomized public key pk_{ID} and a new state St'.
- SFW.Sign$(m, sk_{ID}) \rightarrow \sigma$: is a signing algorithm to sign a message m using an updated secret key sk_{ID}, producing a signature σ of m.
- SFW.Verify$(m, \sigma, pk_{ID}) \rightarrow b \in \{0, 1\}$: is a verification algorithm that takes as input a message m, a signature σ and an updated public key sk_{ID}, and returns 1 or 0.

Notice that the correctness, the unlinkability and the unforgeability of a stateful deterministic wallet scheme SFW are similar for those of stateless deterministic wallet scheme DW (for more details, see [DFL19, AADE+20]).

2.3 Identification schemes and Fiat Shamir Transformation

Definition 15 (Sigma protocol with abort [DFPS23])**.** Let $\mathcal{X}$ and $\mathcal{Y}$ be two finite sets. A $\Sigma-$protocol for a relation $R \subseteq \mathcal{X} \times \mathcal{Y}$ with commitment set $\mathcal{W}$, challenge set $\mathcal{C}$ and response set $\mathcal{Z}$ is a 3-round interactive proof system between a prover written as $Prv = (Prv_1, Prv_2)$ and a verifier $Vrf = (Vrf_1, Vrf_2)$ with the following specifications:

- $\mathrm{Prv}_1 : (x, y) \mapsto (w, st)$ is a PPT algorithm that takes as input a pair of strings in $\mathcal{X} \times \mathcal{Y}$ and outputs a commitment $w \in \mathcal{W}$ and a state $st \in \{0, 1\}^\star$;
- $\mathrm{Vrf}_1 : (x, w) \mapsto c$ is a PPT algorithm that takes as inputs a string $x \in \mathcal{X}$ and a commitment $w \in \mathcal{W}$ and outputs a challenge $c \in \mathcal{C}$;
- $\mathrm{Prv}_2 : (x, y, w, c, st) \mapsto z$ is a PPT algorithm that takes as input a pair of strings in $\mathcal{X} \times \mathcal{Y}$, a commitment $w \in \mathcal{W}$, a challenge $c \in \mathcal{C}$ and a state $st \in \{0, 1\}^\star$ and outputs a response $z \in \mathcal{Z} \cup \{\perp\}$ (we say that Prv_2 aborts if it outputs $\perp$);
- $\mathrm{Vrf}_2 : (x, w, c, z) \mapsto c$ is a deterministic polynomial-time algorithm that takes as inputs a string $x \in \mathcal{X}$ and a commitment $w \in \mathcal{W}$, a challenge $c \in \mathcal{C}$ and a response $z \in \mathcal{Z}$ and outputs a bit b which represents acceptance or rejection; in the case that $z = \perp$, it returns 0.

Definition 16 (Correctness of a sigma protocol [DFPS23])**.** Let $\delta_1, \delta_2 > 0$. A $\Sigma-$protocol $((\mathrm{Prv}_1, \mathrm{Prv}_2), (\mathrm{Vrf}_1, \mathrm{Vrf}_2))$ is $(\delta_1, \delta_2)-$correct if for every $x \in \mathcal{X}$ and valid witness $y \in \mathcal{Y}$ the following holds.

- If the response of the prover is not $\perp$, the verifier accepts with probability at least δ_1:

$$Pr\left[\begin{array}{l|c} \mathrm{Vrf}_2(x, w, c, z) = 1 & \begin{array}{c}(w, st) := \mathrm{Prv}_1(x, y),\\ c := \mathrm{Vrf}_1(x, w),\ z := \mathrm{Prv}_2(x, y, w, c, st),\\ z \neq \perp\end{array}\end{array}\right] \geq \delta_1.$$

- The probability that the prover aborts is bounded by δ_2:

$$Pr\left[\begin{array}{l|c} z = \perp & \begin{array}{c}(w, st) := \mathrm{Prv}_1(x, y),\\ c := \mathrm{Vrf}_1(x, w),\ z := \mathrm{Prv}_2(x, y, w, c, st),\end{array}\end{array}\right] \leq \delta_2.$$

Definition 17 (Identification schemes [DFPS23])**.** An identification scheme (protocol) is a $\Sigma-$protocol for an NP relation $\mathcal{R}$, where the prover and verifier are dealt their statement and witness by a PPT instance generator $\mathrm{IGen}(1^\lambda) \rightarrow (x, y)$.

Definition 18 (Commitment-recoverable IDS [BBD+23])**.** An identification scheme (IDS) is commitment-recoverable, if for any $(x, y) \in \text{IGen}(1^\lambda)$, $c \in \mathcal{C}$, and $z \in \mathcal{Z}$, there exists a unique $w \in \mathcal{W}$ such that $\text{Vrf}_2(x, w, c, z) = 1$. This unique w can be publicly computed using a commitment recovery algorithm as $w := \text{Recover}(x, c, z)$.

Definition 19 (No-Abort Statistical Honest-Verifier Zero-Knowledge [DFPS23])**.** Let $\epsilon_{zk}, T > 0$. A $\Sigma-$protocol is (ϵ_{zk}, T)-naHVZK if there exists a simulator Sim with runtime at most T, that given x, outputs a transcript (w, c, z) such that the distribution of (w, c, z) has statistical distance at most ϵ_{zk} from a honestly generated transcript (w_0, c_0, z_0) produced by the interaction conditioned on $z \neq \perp$.

Definition 20 (Commitment Min-Entropy [DFPS23])**.** For α, we say that an identification scheme $IDS := ((\text{Prv}_1, \text{Prv}_2), (\text{Vrf}_1, \text{Vrf}_2))$ with instance generator $\text{IGen}(1^\lambda)$ has commitment min-entropy α if

$$H_\infty[w : (w, st) \leftarrow \text{Prv}_1(x, y)] \geq \alpha \quad \text{for all } (x, y) \leftarrow \text{IGen}(1^\lambda).$$

Definition 21 (FS Transformation, adapted from [DFPS23, BBD+23])**.** Let $IDS := ((\text{Prv}_1, \text{Prv}_2), (\text{Vrf}_1, \text{Vrf}_2))$ a commitment-recoverable identification scheme. Let $H : \{0,1\}^\star \to \mathcal{C}$ be a hash function where $\mathcal{C}$ is the challenge space of the IDS. Let Recover a function that takes as inputs a challenge $c \in \mathcal{C}$, a response $z \in \mathcal{Z} \cup \{\perp\}$ and outputs a commitment $w \in \mathcal{W}$. One can construct a signature scheme via a Fiat-Shamir transformation with unbounded aborts (FSwUbA) $FS_\infty[IDS, H]$ from the IDS as in the following figure.

$\underline{\text{KGen}(1^\lambda) :}$
1: $(x, y) \leftarrow \text{IGen}(1^\lambda)$;
2: $pk := x, sk := (x, y)$;
3: **return** (pk, sk)

$\underline{\text{Verify}(m, \sigma = (c, z), pk) :}$
1: $w := \text{Recover}(pk, c, z)$;
2: **return** $\text{Vrf}_2(pk, w, z) \wedge [c = H(m \parallel w)]$

$\underline{\text{Sign}(m, sk) :}$
1: $z := \perp$;
2: **while** $z = \perp$ **do**
3: $\quad (w, st) := \text{Prv}_1(sk)$;
4: $\quad c := H(m \parallel w)$;
5: $\quad z := \text{Prv}_2(sk, w, c, st)$;
6: **end while**
7: **return** $\sigma = (c, z)$

Figure 6: Fiat-Shamir transformation with unbounded aborts (FSwUbA) from a commitment-recoverable identification scheme $IDS := ((\text{Prv}_1, \text{Prv}_2), (\text{Vrf}_1, \text{Vrf}_2))$.

2.4 Summary of Dilithium

CRYSTALS-Dilithium [DKL+18, BDK+21] is a digital signature scheme whose security is related to the Module Learning With Errors (**MLWE**), Module Short Integer Solution (**MSIS**) problems which are standard lattice problems, and a non standard problem called the **SelfTargetMSIS**. Dilithium is one of the winner of National Institute of Standards and Technology (NIST) competition [Nat24]. We summarize, in a high level of description, the Dilithium algorithms and the fundamental functions used into these algorithms.

Notations. we introduce some notation and recall a couple of elementary functions which will be useful in the rest of this paper.

Rings and Probability distributions: Let k, l and d be positive integer parameters. Let $R = \mathbb{Z}/\langle x^n + 1\rangle, R_q = \mathbb{Z}_q/\langle x^n + 1\rangle$ for $n = 256$ and $q = 8380417 = 2^{23} - 2^{13} + 1$. For a finite set F, we let $\mathcal{U}(F)$ denote the uniform distribution over F. For a distribution D, we let $x \leftarrow D$ denote the element x is sampled randomly from the distribution D. When D is a finite set, we will write $x \leftarrow D$ or $x \leftarrow \mathcal{U}(D)$ to mean x is sampled uniformly at

random from D.

Integer Arithmetic: For an odd (resp. even) positive integer α, we define $r' = r \bmod^{\pm} \alpha$ to be the unique element r' in the range $-\frac{\alpha-1}{2} \leq r' \leq \frac{\alpha-1}{2}$ (resp. $-\frac{\alpha}{2} < r' \leq \frac{\alpha}{2}$) such that $r' \equiv r \mod \alpha$. For a positive integer α, we define $r' = r \bmod^{+} \alpha$ to be the unique element r' in the range $0 \leq r' < \alpha$ such that $r' \equiv r \mod \alpha$.

Norm, Sets and Parameters: For an element $w \in \mathbb{Z}_q$, $\| w \|_\infty = |w \bmod^{\pm} q|$ and for $w = \sum_{i=0}^{n-1} w_i x^i \in R$, $\| w \|_\infty = \max_i \| w_i \|_\infty$ and $\| w \| = \sqrt{\sum_i \| w_i \|}$, and similarly for $w \in (w_1, w_2, \ldots, w_k) \in R^k$, we define $\| w \|_\infty = \max_i \| w_i \|_\infty$ and $\| w \| = \sqrt{\sum_i \| w_i \|}$. We denote by S_η the set of elements w in R_q such that $\| w \|_\infty \leq \eta$ for a positive integer η. Let B_τ denote the set of elements of R that have τ coefficients that are either -1 or 1 and the rest are 0 i.e $B_\tau = \{c \in R : \| c \|_\infty = 1 \text{ and } \| c \| = \sqrt{\tau}\}$. From τ and η, we derive the parameter $\beta = \tau \cdot \eta$.

Functions and Mappings: The function **ExpandA** maps a uniform seed $\rho \in \{0,1\}^{256}$ to a matrix $A \in R_q^{k \times l}$ in the Number Theoretic Transform (NTT) domain representation. **ExpandMask** maps a seed ρ' and a nonce κ to $y \in S_{\gamma_1}^l$. The function **ExpandS** maps a seed ρ' to $(s_1, s_2) \leftarrow \mathcal{U}(S_\eta^l) \times \mathcal{U}(S_\eta^k)$. The function **CRH** is a collision resistant hash function mapping to $\{0,1\}^{384}$, and H is the extended output function (XOF) mapping onto the specified domain and is instantiated with SHAKE-256 in Dilithium. The function ExpandSP is a hash function that returns uniform elements of $S_\eta^l \times S_\eta^k$. When a matrix $A \in R_q^{k \times l}$ is generated by a seed, we will sometimes denote this seed by $\rho(A)$.

In the following Figure 7, we give the functions used in Dilithium key generation, signing and verification algorithms.

Power2Round$_q(r, d)$
1: $r := r \bmod^{+} q$;
2: $r_0 := r \bmod^{\pm} 2^d$; $r_1 := (r - r_0)/2^d$;
3: **return** (r_1, r_0)

MakeHint$_q(z, r, \alpha)$
1: $r_1 := \mathbf{HighBits}_q(r, \alpha)$;
2: $v_1 := \mathbf{HighBits}_q(r + z, \alpha)$;
3: **return** $(r_1 \neq v_1)$

Decompose$_q(r, \alpha)$
1: $r := r \bmod^{+} q$; $r_0 := r \bmod^{\pm} \alpha$;
2: **if** $r - r_0 = q - 1$ **then return** $(0, r_0 - 1)$;
3: **else** $r_1 := (r - r_0)/\alpha$; **return** (r_1, r_0);

HighBits$_q(r, \alpha)$
1: $(r_1, r_0) := \mathbf{Decompose}_q(r, \alpha)$; **return** r_1

LowBits$_q(r, \alpha)$
1: $(r_1, r_0) := \mathbf{Decompose}_q(r, \alpha)$; **return** r_0

UseHint$_q(h, r, \alpha)$
1: $m := (q - 1)/\alpha$; $(r_1, r_0) := \mathbf{Decompose}_q(r, \alpha)$;
2: **if** $h = 1$ and $r_0 > 0$ **return** $(r_1 + 1) \bmod^{+} m$;
3: **if** $h = 1$ and $r_0 \leq 0$ **return** $(r_1 - 1) \bmod^{+} m$
4: **return** r_1

Figure 7: Supporting algorithms for Dilithium scheme.

The **HighBits**$_q$ function verifies the following property.

Lemma 2 ([DKL+18]). *If* $\| s \|_\infty \leq \beta$ *and* $\| \mathbf{LowBits}_q(r, \alpha) \|_\infty \leq \alpha/2 - \beta$, *then*

$$\mathbf{HighBits}_q(r, \alpha) = \mathbf{HighBits}_q(r + s, \alpha).$$

In the following Figure 8, we recall the different algorithms for Dilithium signature scheme such as key generation, signature and verification algorithms presented in the third

round of NIST which we call Dilithium V.1.3.

Dilithium.KGen() :

$\zeta \leftarrow \{0,1\}^{256}$;
$(\rho, \rho', K) := H(\zeta)$;
$(s_1, s_2) \in S_\eta^l \times S_\eta^k := \mathbf{ExpandS}(\rho')$;
$\mathbf{A} \sim R_q^{k \times l} := \mathbf{ExpandA}(\rho)$;
$t := \mathbf{A}s_1 + s_2$;
$(t_1, t_0) := \mathbf{Power2Round}_q(t, d)$;
$tr \in \{0,1\}^{384} := \mathbf{CRH}(\rho \parallel t_1)$;
$pk := (\rho, t_1)$;
$sk := (\rho, K, tr, s_1, s_2, t_0)$;
return (pk, sk)

Dilithium.Verify$(m, sk = (\rho, t_1), \sigma = (z, h, c))$:

$\mathbf{A} \sim R_q^{k \times l} := \mathbf{ExpandA}(\rho)$;
$tr \in \{0,1\}^{384} := \mathbf{CRH}(\rho \parallel t_1)$;
$\mu \in \{0,1\}^{384} := \mathbf{CRH}(tr \parallel m)$;
$c := \mathbf{SampleInBall}(c)$;
$w_1' := \mathbf{UseHint}_q(h, \mathbf{A}z - ct_1 \cdot 2^d, 2\gamma_2)$;
$b_1 := [\parallel z \parallel_\infty \leq \gamma_1 - \beta]$;
$b_2 := [c = H(\mu \parallel w_1')]$;
$b_3 := [\#$ of 1's **h** is $\leq \omega]$;
return $b_1 \wedge b_2 \wedge b_3$

Dilithium.Sign$(m, pk = (\rho, K, tr, s_1, s_2, t_0))$:

$\mathbf{A} \sim R_q^{k \times l} := \mathbf{ExpandA}(\rho)$;
$\mu \in \{0,1\}^{384} := \mathbf{CRH}(tr \parallel m)$;
$\kappa := 0; (z, h) := \perp$;
$\rho' \in \{0,1\}^{512} := H(K \parallel \mu)$; ▷ (or $\rho' \leftarrow \{0,1\}^{512}$)
while $(z, h) = \perp$ **do**
  $y \in S_{\gamma_1 - 1}^l := \mathbf{ExpandMask}(\rho' \parallel \kappa)$;
  $w := \mathbf{A}y$;
  $w_1 := \mathbf{HighBits}_q(w, 2\gamma_2)$;
  $c \in \{0,1\}^{256} := H(\mu \parallel w_1)$;
  $c \in B_{60} := \mathbf{SampleInBall}(c)$;
  $z := y + cs_1$;
  $r_0 := \mathbf{LowBits}_q(w - cs_2, 2\gamma_2)$;
  if $\parallel z \parallel_\infty \geq \gamma_1 - \beta$ or $\parallel r_0 \parallel_\infty \geq \gamma_2 - \beta$ **then**
    $(z, h) := \perp$;
  else
    $h := \mathbf{MakeHint}_q(-ct_0, w - cs_2 + ct_0, 2\gamma_2)$;
    nb1h := [the # of 1's in **h** is greater than ω];
    if $\parallel ct_0 \parallel_\infty > \gamma_2$ or nb1h **then**
      $(z, h) := \perp$;
    end if
    $\kappa := \kappa + 1$
  end if
end while
return $\sigma = (z, h, c)$

Figure 8: Dilithium : key generation, signature and verification algorithms.

3 Our constructions

The key idea to build a signature scheme with reradomizable keys is to start from an existing signature scheme which has additional properties such as key homomorphism.

Let $\Sigma = (\Sigma.\text{KGen}, \Sigma.\text{Sign}, \Sigma.\text{Verify})$ be a signature scheme with the following properties. There exists a large set $\mathcal{S}$ and a function $f : (x, y) \mapsto f(x, y)$ such that for any $(pk, sk) \leftarrow \Sigma.\text{KGen}(1^\lambda)$ and $\mu \leftarrow \mathcal{S}$, the following holds

$$\Pr[\Sigma.\text{Verify}\,(m, \Sigma.\text{Sign}(m, sk + \mu), f(pk, \mu)) = 1] \geq 1 - \delta$$

for a negligible δ and for all messages m in the message space. We then construct a signature scheme with rerandomizable keys $\Sigma_{RK} = (\Sigma_{\text{RK}}.\text{KGen}, \Sigma_{\text{RK}}.\text{Sign}, \Sigma_{\text{RK}}.\text{Verify}, \Sigma_{\text{RK}}.\text{UpdateSk}, \Sigma_{\text{RK}}.\text{UpdatePk})$ as follows

- $\Sigma_{\text{RK}}.\text{KGen}(1^\lambda) := \Sigma.\text{KGen}(1^\lambda) \rightarrow (pk, sk)$
- $\Sigma_{\text{RK}}.\text{Sign}(m, sk) := \Sigma.\text{Sign}(m, sk) \rightarrow \sigma$
- $\Sigma_{\text{RK}}.\text{Verify}(m, \sigma, pk) := \Sigma.\text{Verify}(m, \sigma, pk) \in \{0, 1\}$
- $\Sigma_{\text{RK}}.\text{UpdateSk}(sk, \mu) \rightarrow sk' = sk + \mu$
- $\Sigma_{\text{RK}}.\text{UpdatePk}(pk, \mu) \rightarrow pk' = f(pk, \mu)$

If the signature scheme is based on lattice problems such as module learning with errors (including Ring-LWE and LWE), the function f can be constructed as follows.

Suppose that $pk = g(sk)$ for some one way function g which is almost homomorphic i.e $g(sk_1 + sk_2) = g(sk_1) + g(sk_2) + e$ for some error $e \in \chi$. Then we can define f as follows : $f(pk, \mu) = pk + g(\mu)$.

In general case, we need a one way group homomorphic function g in the form $g : (\mathbb{H}, \circ) \mapsto (\mathbb{G}, \cdot) : (x \circ y) = g(x) \cdot g(y)$. In this case if $pk = g(sk)$, for an update ρ, we will have $pk \cdot g(\rho) = g(sk) \cdot g(\rho) = g(sk \circ \rho)$. In this case, one can take as new session public key $pk' = pk \cdot g(\rho)$ and new session private key $sk' = sk \circ \rho$.

For instance, for signature schemes in which the security of the private keys is related to

i) Module-LWE problem: $g(s, e) = As + e$ for $s \in \chi_s$ and $e \in \chi_e$.

ii) Discrete logarithm problem : $g(a) = g^a$ for g a generator of some cyclic group G noted multiplicatively.

3.1 Our proposed signature schemes with rerandomized keys based on Dilithium

In this subsection, we present our main signature schemes with rerandomized keys based on Dilithium (V.1.3) with uncompressed public keys. In Dilithium, the uncompressed public key takes the form $pk = (\mathbf{A}, t)$ where $t = \mathbf{A}s_1 + s_2$. If we update the public key to the form $(\mathbf{A}, t')$ with $\mathbf{A}$ randomly and uniformly generated for each user, it becomes possible to link two updated public keys from different master public keys. For instance, let $(\mathbf{A}, t)$ and $(\mathbf{B}, \widehat{t})$ be two master public keys, and $(\mathbf{A}, t')$ (resp. $(\mathbf{B}, \widehat{t'})$) be the update from $(\mathbf{A}, t)$ (resp. $(\mathbf{B}, \widehat{t})$). These two session public keys are linkable based on the difference between the first components ($\mathbf{B} \neq \mathbf{A}$). To achieve a crucial security property of the signature scheme with randomizable keys from Dilithium, which is unlinkability, we assume dealing with a network $\mathcal{N}$ comprising subnetworks $\mathcal{N}_1, \mathcal{N}_2, \ldots, \mathcal{N}_r$. Each subnetwork contains a set of distinct master keypairs, and within each subnetwork $\mathcal{N}_i$, a common matrix $\mathbf{A}_i$ randomly and uniformly generated is shared. In this setup, the network $\mathcal{N}$ has a database of matrices randomly and uniformly generated, and users generating a master keypair select a matrix or seed from this database. In such a network, the unlinkability property only concerns a subnetwork $\mathcal{N}_i$, as long as this subnetwork is sufficiently large. Thus, for the remainder of this paper, we assume we are dealing with a single subnetwork. For our signature scheme with rerandomized keys based on Dilithium, all master keys share the same seed $\rho \in \{0, 1\}^s, s \geq 128$. For the signature scheme with rerandomized keys based on the simplified Dilithium (Dilithium with uncompressed public keys), all master keys share the same matrix $\mathbf{A} \leftarrow \mathcal{U}(R_q^{k \times l})$. We then define two functions : **getSeedRhoFromDB**() which returns the seed ρ and **getMatrixAFromDB**() which returns the matrix $\mathbf{A} \in R_q^{k \times l}$.

DilithiumRK. We present in the following figures (Fig. 9 and Fig. 10), DilithiumRK, our main signature scheme with rerandomizable keys based on Dilithium with uncompressed public keys.

DilithiumRK.KGen() :

$(s_1, s_2) \leftarrow \mathcal{U}(S_\eta^l) \times \mathcal{U}(S_\eta^k)$;
$\mathbf{A} := \mathbf{getMatrixAFromDB}()$;
$t := \mathbf{A}s_1 + s_2$;
$pk := (\mathbf{A}, t)$; $sk := (\mathbf{A}, s_1, s_2)$;
return (pk, sk)

DilithiumRK.Verify(m, pk, σ) :

$(\mathbf{A}, t) := pk$; $(c, z) := \sigma$;
$r'_0 := \mathbf{LowBits}_q(\mathbf{A}z - ct, 2\gamma_2)$;
$w'_1 := \mathbf{HighBits}_q(\mathbf{A}z - ct, 2\gamma_2)$;
$c' := H(m \parallel w'_1)$;
$b_0 := [\![\parallel r'_0 \parallel_\infty < \gamma_2 - \beta]\!]$;
$b_1 := [\![\parallel z \parallel_\infty < \gamma_1 - \beta]\!]$;
$b_2 := [c = c']$;
return $b_0 \wedge b_1 \wedge b_2$

DilithiumRK.Sign$(m, sk = (\mathbf{A}, s_1, s_2))$:

$r := \perp$;
while $r = \perp$ **do**
  $y \leftarrow \mathcal{U}(S^l_{\gamma_1 - 1})$;
  $w := \mathbf{A}y$;
  $w_1 := \mathbf{HighBits}_q(w, 2\gamma_2)$;
  $c \in B_\tau := H(m \parallel w_1)$;
  $z := y + cs_1$;
  $r_0 := \mathbf{LowBits}_q(w - cs_2, 2\gamma_2)$;
  if $\parallel z \parallel_\infty < \gamma_1 - \beta$ and $\parallel r_0 \parallel_\infty < \gamma_2 - \beta$ **then**
    $r := z$;
  end if
end while
return $\sigma = (c, r = z)$

Figure 9: Algorithms of DilithiumRK and the Dilithium signature with uncompressed public keys. For DilithiumRK, the parameter β is set such that $\beta \geq \max_{s \in S_{2\eta}, c \in B_\tau} \parallel cs \parallel_\infty$ and for Dilithium we have $\beta \geq \max_{s \in S_\eta, c \in B_\tau} \parallel cs \parallel_\infty$.

DilithiumRK.UpdatePk(pk, μ) :

$(\mathbf{A}, t) := pk$;
$(s'_1, s'_2) := \mathrm{ExpandSP}(\rho(\mathbf{A}) \parallel \mu)$;
$t' := \mathbf{A}s'_1 + s'_2$; $t'' := t + t'$
$pk' := (\mathbf{A}, t'')$;
return pk'

DilithiumRK.UpdateSk(sk, μ) :

$(\mathbf{A}, s_1, s_2) := sk$;
$(s'_1, s'_2) := \mathrm{ExpandSP}(\rho(\mathbf{A}) \parallel \mu)$;
$s''_1 := s_1 + s'_1$; $s''_2 := s_2 + s'_2$;
$sk' := (\mathbf{A}, s''_1, s''_2)$;
return sk'

Figure 10: Public and private keys update algorithms of DilithiumRK.

Notice that in the verification algorithm, we have added an additional condition to check i.e $\parallel \mathbf{LowBits}_q(\mathbf{A}z - ct, 2\gamma_2) \parallel_\infty < \gamma_2 - \beta$. It is not necessary for the correctness but it is used in the proof of unforgeability of DilithiumRK in Sect. 4.2.

Correctness. Let us show that DilithiumRK is correct. Suppose that

$$\beta \geq \max_{s \in S_{2\eta}, c \in B_\tau} \parallel cs \parallel_\infty$$

- The first requirement of correctness is satisfied (it follows from Dilithium correctness) i.e. For all $(pk_0, sk_0) \leftarrow \mathrm{DilithiumRK.KGen}(1^\lambda)$ and all $m \in \{0,1\}^*$ we have that

$$\mathrm{DilithiumRK.Verify}\left(m, \mathrm{DilithiumRK.}Sign(m, sk_0), pk_0\right) = 1.$$

- Let us now prove the second condition of the correctness i.e.
For $(pk_0, sk_0) \leftarrow \mathrm{DilithiumRK.KGen}(1^\lambda)$, $pk_i \leftarrow \mathrm{DilithiumRK.UpdatePk}(pk_0, \mu)$ and $sk_i \leftarrow \mathrm{DilithiumRK.UpdateSk}(sk_0, \mu)$, we have the following.

$$\mathrm{DilithiumRK.Verify}\left(m, \mathrm{DilithiumRK.Sign}(m, sk_i), pk_i\right) = 1.$$

Proof. Let $\sigma = (c, z)$ be a valid signature on a message m with the private key $sk = (\mathbf{A}, s_1, s_2)$ with $s_1 = s'_1 + s''_1$ and $s_2 = s'_2 + s''_2$. To demonstrate that the verification of this signature with the public key $pk = (\mathbf{A}, t)$ will yield a result of 1, we need to establish that $H(m \parallel w'_1) = H(m \parallel w_1)$ holds. To achieve this equality, we will prove that $w_1 = w'_1$ where $w'_1 = \mathbf{HighBits}_q(Az - ct, 2\gamma_2)$ and $w'_1 = \mathbf{HighBits}_q(Ay, 2\gamma_2)$. We have the following

$$\begin{aligned}\mathbf{A}z - ct &= \mathbf{A}z - c\left(\mathbf{A}(s_1' + s_1'') + (s_2' + s_2'')\right)\\ &= \mathbf{A}(z - c(s_1' + s_1'')) - c(s_2' + s_2'')\\ &= \mathbf{A}(z - cs_1) - c(s_2' + s_2'')\\ &= \mathbf{A}y - c(s_2' + s_2'')\\ &= \mathbf{A}y - cs_2\end{aligned}$$

This show that

$$w_1' = \mathbf{HighBits}_q(Az - ct, 2\gamma_2) = \mathbf{HighBits}_q(Ay - cs_2, 2\gamma_2)$$

Since $\sigma = (c, z)$ is a valid signature, we have $\| \mathbf{LowBits}_q(w - cs_2, 2\gamma_2) \|_\infty < \gamma_2 - \beta$. Then by Lemma 2, we can conclude that $w_1' = w_1$ since the parameter β is chosen such that $\| cs_2 \|_\infty \leq \beta$. Furthermore,

$$\| \mathbf{LowBits}_q(\mathbf{A}z - ct, 2\gamma_2) \|_\infty = \| \mathbf{LowBits}_q(\mathbf{A}y - cs_2, 2\gamma_2) \|_\infty < \gamma_2 - \beta$$

Therefore $b_0 \wedge b_1 \wedge b_2 = 1$. □

3.2 Our Proposed Deterministic Wallets

Using our signature scheme with rerandomizable keys DilithiumRK, we build a stateless deterministic wallet DilithiumDW in the following Figure 11.

DilithiumDW.KGen(1^λ) :
1: $(\mathbf{A}, t), (\mathbf{A}, s_1, s_2) \leftarrow$ DilithiumRK.KGen(1^λ);
2: $tr := \mathbf{CRH}(\rho(A) \parallel t)$; $ch \leftarrow \{0,1\}^\lambda$
3: $mpk := (\mathbf{A}, t)$; $msk := (\mathbf{A}, tr, s_1, s_2)$;
4: **return** (mpk, msk, ch)

DilithiumDW.Verify(m, σ, pk_{ID}) :
1: $(\mathbf{A}, t) := pk_{ID}$;
2: $tr := \mathbf{CRH}(\rho(A) \parallel t)$; $\mu := \mathbf{CRH}(tr \parallel m)$;
3: $b :=$ DilithiumDW.Verify(μ, σ, pk_{ID});
4: **return** b

DilithiumDW.UpdateSk(msk, ρ_{ID}) :
1: $(\mathbf{A}, s_{1ID}, s_{2ID}) :=$ DilithiumRK.UpdateSk(msk, ρ_{ID});
2: $t_{ID} := \mathbf{A}s_{1ID} + s_{2ID}$; $tr_{ID} := \mathbf{CRH}(\rho(A) \parallel t_{ID})$;
3: **return** $sk_{ID} := (\mathbf{A}, tr_{ID}, s_{1ID}, s_{2ID})$

DilithiumDW.Sign(m, sk_{ID}) :
1: $(\mathbf{A}, tr, s_1, s_2) := sk_{ID}$;
2: $\mu := \mathbf{CRH}(tr \parallel m)$;
3: $\sigma :=$ DilithiumRK.Sign($\mu, sk = (\mathbf{A}, s_1, s_2)$);
4: **return** σ

DilithiumDW.RandGen(ch, ID) :
1: $\rho_{ID} := \mathbf{CRH}(ch \parallel ID)$
2: **return** ρ_{ID}

DilithiumDW.UpdatePk(mpk, ρ_{ID}) :
1: $pk_{ID} :=$ DilithiumRK.UpdatePk(mpk, ρ_{ID});
2: **return** pk_{ID}

Figure 11: Key generation, signing, verification, public and private keys update algorithms of our stateless deterministic wallet DilithiumDW.

Similarly, using the generic construction of Das et al. (Fig.7 [DFL19]), we define a stateful deterministic wallet DilithiumSDW from our signature scheme with rerandomized keys DilithiumRK in the following Figure 12. Let $H : \{0,1\}^\star \to \{0,1\}^n \times \{0,1\}^\lambda$ a collision resistant hash function.

DilithiumSDW.KGen(1^λ) :

1: $(\mathbf{A}, t), (\mathbf{A}, s_1, s_2) \leftarrow$ DilithiumRK.KGen(1^λ);
2: $tr := \mathbf{CRH}(\rho(A) \parallel t)$; $St_0 \leftarrow \{0,1\}^\lambda$
3: $mpk := (\mathbf{A}, t)$; $msk := (\mathbf{A}, tr, s_1, s_2)$;
4: **return** (mpk, msk, St_0)

DilithiumSDW.Sign(m, sk_{ID}) :

1: $(\mathbf{A}, tr, s_1, s_2) := sk_{ID}$;
2: $\mu := \mathbf{CRH}(tr \parallel m)$;
3: $\sigma :=$ DilithiumRK.Sign($\mu, sk = (\mathbf{A}, s_1, s_2)$);
4: **return** σ

DilithiumSDW.Verify(m, σ, pk_{ID}) :

1: $(\mathbf{A}, t) := pk_{ID}$;
2: $tr := \mathbf{CRH}(\rho(A) \parallel t)$; $\mu := \mathbf{CRH}(tr \parallel m)$;
3: $b :=$ DilithiumDW.Verify(μ, σ, pk_{ID});
4: **return** b

DilithiumSDW.UpdatePk(mpk, ID, St_i) :

1: $\rho_{ID}, St_{i+1} := H(ID \parallel St_i)$;
2: $pk_{ID} :=$ DilithiumRK.UpdatePk(mpk, ρ_{ID});
3: **return** (pk_{ID}, St_{i+1})

DilithiumSDW.UpdateSk(msk, ID, St_i) :

1: $\rho_{ID}, St_{i+1} := H(ID \parallel St_i)$;
2: $(\mathbf{A}, s_{1ID}, s_{2ID}) :=$ DilithiumRK.UpdateSk(msk, ρ_{ID});
3: $t_{ID} := \mathbf{A}s_{1ID} + s_{2ID}$; $tr_{ID} := \mathbf{CRH}(\rho(A) \parallel t_{ID})$;
4: $sk_{ID} := (\mathbf{A}, tr_{ID}, s_{1ID}, s_{2ID})$
5: **return** (sk_{ID}, St_{i+1})

Figure 12: Key generation, signing, verification, public and private keys update algorithms of our stateful deterministic wallet DilithiumSDW.

4 Security Analysis

In this section, we will begin by analyzing the hard problems related to our scheme, followed by the proof of unforgeability and unlinkability.

4.1 Hard Problems related to our Scheme

Let m and k be two positive integers. Let χ be a distribution on R_q. We will start by search version of module learning with errors variants.

Definition 22 ($\mathbf{S-MLWE}_\chi$ [LS15])**.** The search version of Module Learning With Errors $\mathbf{S-MLWE}_\chi$ asks to find (s, e) when given $(\mathbf{A}, \mathbf{A}s + e)$ for $\mathbf{A} \leftarrow \mathcal{U}(R_q^{m \times k})$, and $(s, e) \leftarrow \chi^k \times \chi^m$.

The goal of the decision version of the Module Learning With Errors for an adversary $\mathcal{A}$ is to distinguish between $(\mathbf{A}, \mathbf{A}s + e)$ and $(\mathbf{A}, \mathbf{u})$ for $\mathbf{A} \leftarrow \mathcal{U}(R_q^{m \times k})$, $(s, e) \leftarrow \chi^k \times \chi^m$, and $\mathbf{u} \leftarrow \mathcal{U}(R_q^m)$. We define the advantage of $\mathcal{A}$ in the $\mathbf{D-MLWE}_\chi$ game as

$$\mathrm{Adv}_\chi^{\mathbf{D-MLWE}}(\mathcal{A}) = |Pr\left[\mathcal{A}(\mathbf{A}, \mathbf{A}s + e) \rightarrow 1\right] - Pr\left[\mathcal{A}(\mathbf{A}, \mathbf{u}) \rightarrow 1\right]|.$$

Both variants are shown to be at least as hard as solving an hard problem on Module lattices [LS15] when the distribution of the error follows a discrete Gaussian distribution. Other distributions of the error and the secret are studied in [BJRW23].

In our scheme, the secrets and errors are a sum of two independent elements which corresponds to have a $\mathbf{S-MLWE}$ instance of the form $(\mathbf{A}, \mathbf{A}(s + s') + (e + e'))$. This instance is at least as hard as the original $\mathbf{S-MLWE}$ problem as given $(\mathbf{A}, b = \mathbf{A}s + e)$, an adversary $\mathcal{B}$ samples two elements (s', e') from the distribution $\chi^k \times \chi^m$ and forwards $(\mathbf{A}, \mathbf{A}(s + s') + (e + e'))$ to the corresponding oracle. Given the secret $(s + s')$, $\mathcal{B}$ can easily solve the $\mathbf{S-MLWE}$ problem.

We use the notation $\mathbf{S-MLWE}_{\mathbf{A},\chi}$ (resp. $\mathbf{D-MLWE}_{\mathbf{A},\chi}$) to refer as the $\mathbf{S-MLWE}_\chi$ (resp. $\mathbf{D-MLWE}_{\mathbf{A},\chi}$) problem when the matrix $\mathbf{A}$ is shared with a network. Even if an

adversary has access to a hot stateless wallet and thus knows the session public keys, it remains computationally hard to recover the session secret keys, assuming the hardness of the $\mathbf{S}-\mathbf{MLWE}_{\chi}$ problem. Notice that a session public key $t' = \mathbf{A}(s+s')+(e+e')$ is completely random and uniformly distributed in R_q^m assuming the hardness of $\mathbf{D}-\mathbf{MLWE}_{\mathbf{A},\chi}$ since $\mathbf{A}(s+s')+(e+e') = (\mathbf{A}s+e)+(\mathbf{A}s'+e')$ is a sum of two uniform elements $(u, u') \in R_q^m \times R_q^m$.

In the following two subsections, we will comprehensively analyze the unforgeability and unlinkability aspects of our wallets. It is shown that if a deterministic wallet is constructed from a signature scheme with rerandomizable keys (see Fig. 6 in [Hu23] and Fig. 7 in [DFL19]) that satisfies unlinkability and unforgeability properties, then the corresponding deterministic wallet also inherits these properties (see [Hu23, DFL19, AADE+20]). Therefore, our focus will be on showing that our signature scheme with rerandomizable keys DilithiumRK satisfies both unlinkability and unforgeability properties.

4.2 Unforgeability of DilithiumRK

In the following, we will show that the unforgeability of our DilithiumRK follows from the unforgeability of Dilithium i.e if there exists an adversary against DilithiumRK, then one can use this adversary to build another adversary against Dilithium with uncompressed public keys.

Theorem 1. *The signature scheme with rerandomizable keys DilithiumRK is* ***Uf-CMA-HRK*** *secure in the (Q)ROM if the signature Dilithium with uncompressed public keys is* ***Uf-CMA*** *secure in the (Q)ROM.*

Proof. Let $\mathcal{A}$ be an adversary capable of generating valid forgeries under the signature scheme with rerandomizable keys DilithiumRK. In other words, $\mathcal{A}$ can win the game $\mathbf{Uf\text{-}CMA\text{-}HRK}_{\text{DilithiumRK}}$ defined in Figure 2. We construct an algorithm $\mathcal{B}$ that runs $\mathcal{A}$ as a subroutine to win the game $\mathbf{Uf\text{-}CMA}_{\text{Dilithium}}$, defined in Figure 1, against the Dilithium signature scheme with uncompressed public keys. According to the security model, the adversary $\mathcal{A}$ has access to a random oracle $\mathcal{O}$Rand as well as a signing oracle $\mathcal{O}\text{Sign}_{\text{DilithiumRK}}$, which provides signatures from the DilithiumRK signature scheme. The reduction $\mathcal{B}$ has access to the signing oracle $\mathcal{O}\text{Sign}_{\text{Dilithium}}$, which returns signatures generated by the Dilithium signature scheme. In this reduction, $\mathcal{B}$ plays two roles: as an adversary against the unforgeability of the Dilithium signature scheme and as the challenger for the adversary $\mathcal{A}$ in its attack against the unforgeability of DilithiumRK. Additionally, $\mathcal{B}$ is challenged by an external challenger. At the start of the reduction, $\mathcal{B}$ initializes two empty lists, MList := $\emptyset$ and RList := $\emptyset$, which are used to record the queries made to the randomness oracle and the signing oracle for DilithiumRK, respectively.

Challengers. An external challenger selects a key pair $(pk = (\mathbf{A}, t), sk = (\mathbf{A}, s_1, s_2)) \leftarrow$ DilithiumRK.KGen(1^{λ}) and provides the public key pk to the adversary $\mathcal{B}$ against Dilithium. Acting as a challenger for $\mathcal{A}$, $\mathcal{B}$ forwards pk as the master public key to $\mathcal{A}$. Note that neither $\mathcal{B}$ nor $\mathcal{A}$ knows the secret key sk. We will design the reduction for $\mathcal{B}$ in such a way that it can simulate the signing oracle for DilithiumRK by utilizing its own external signing oracle.

Simulation of Randomness Queries. When $\mathcal{A}$ queries on the oracle $\mathcal{O}$Rand, $\mathcal{B}$ randomly sample ρ from $\mathcal{R}$, sets RList = RList $\cup \{\rho\}$, and finally forwards ρ to $\mathcal{A}$.

Simulation of Random Oracle Queries. When $\mathcal{A}$ makes queries to a random oracle H', the reduction $\mathcal{B}$ redirects these queries to the corresponding random oracle H to which it has access, and then forwards the responses back to $\mathcal{A}$.

Simulation of Signing Queries. When $\mathcal{A}$ queries the signing oracle $\mathcal{O}\text{Sign}_{\text{DilithiumRK}}$ with input $(pk = (\mathbf{A}, t), m, \rho)$, $\mathcal{B}$ first updates the list MList = MList $\cup \{m\}$ and checks whether $\rho \in$ RList. If ρ is not found, $\mathcal{B}$ aborts. Next, $\mathcal{B}$ computes $(s_1', s_2') := \text{ExpandSP}(\rho(\mathbf{A}), \rho)$ and constructs the randomized public key $pk_\rho = (\mathbf{A}, t' = t + \mathbf{A}s_1' + s_2')$. Afterwards, $\mathcal{B}$ queries its external signing oracle with input pk and m, receiving the signature $\sigma' = (c, z_1')$, i.e., $(c, z_1') \leftarrow \mathcal{O}\text{Sign}_{\text{Dilithium}}(m, pk)$. Then, $\mathcal{B}$ computes $z = z_1' + cs_1'$ and checks whether $\| z \|_\infty < \gamma_1 - \beta$ and $\| \mathbf{LowBits}_q(\mathbf{A}z - ct', 2\gamma_2) \|_\infty < \gamma_2 - \beta$. If these conditions are not satisfied, $\mathcal{B}$ queries its external signing oracle again with the same input (pk, m), obtaining $(c, z_2') \leftarrow \mathcal{O}\text{Sign}_{\text{Dilithium}}(m, pk)$, and computes $z = z_2' + cs_1'$. $\mathcal{B}$ continues this process until it finds some i for which $z = z_i' + cs_1'$ satisfies the bounds $\| z \|_\infty < \gamma_1 - \beta$ and $\| \mathbf{LowBits}_q(\mathbf{A}z - ct', 2\gamma_2) \|_\infty < \gamma_2 - \beta$. Finally, $\mathcal{B}$ outputs the signature $\sigma = (c, z)$ for the message m. Notice that this signature can be verified using the randomized public key $pk_\rho = (\mathbf{A}, t')$ as a valid signature i.e. DilithiumRK.Verify $(m, \sigma, pk_\rho) = 1$.

Exploiting the Forgeries. Suppose that $\mathcal{A}$ finds a valid forgery $(m^\star, \sigma^\star = (c^\star, z^\star), \rho^\star)$. This implies that $m^\star \notin$ MList, $\rho \in$ RList and :

$$\text{DilithiumRK.Verify}\,(m^\star, \sigma^\star, pk^\star) = 1, \tag{1}$$

where

$$pk^\star = \text{DilithiumRK.UpdatePk}(pk, \rho^\star) = (\mathbf{A}, t^\star) \text{ where } t^\star = t + \mathbf{A}s_1^\star + s_2^\star,$$
$$sk^\star = \text{DilithiumRK.UpdateSk}(sk, \rho^\star) = (\mathbf{A}, s_1 + s_1^\star, s_2 + s_2^\star).$$

We now construct $\mathcal{B}$ which outputs a valid forgery for the Dilithium signature scheme under the original public key $pk = (\mathbf{A}, t)$. Algorithm $\mathcal{B}$ computes $z^\circ = z^\star - s_1^\star c^\star$, and outputs $(m^\star, \sigma^\circ = (c^\star, z^\circ))$ as a forgery. Let us now show that $(m^\star, \sigma^\circ = (c^\star, z^\circ))$ is indeed a valid forgery for the Dilithium signature scheme, i.e. Dilithium.Verify $(m^\star, \sigma^\circ, pk) = 1$.

Since $(m^\star, \sigma^\star = (c^\star, z^\star), \rho^\star)$ is a valid forgery for DilithiumRK, by Eq. 1, we have: $\| z^\star \|_\infty < \gamma_1 - \beta$, $\quad c^\star = H(m^\star \| w_1^\star)$, where $w_1^\star = \mathbf{HighBits}_q(\mathbf{A}z^\star - c^\star t^\star, 2\gamma_2)$, and

$$\| \mathbf{LowBits}_q(\mathbf{A}z^\star - c^\star t^\star, 2\gamma_2) \|_\infty < \gamma_2 - \beta. \tag{2}$$

We have:

$$\| z^\circ \|_\infty = \| z^\star - c^\star s_1^\star \|_\infty \leq \| z^\star \|_\infty + \| c^\star s_1^\star \|_\infty \leq (\gamma_1 - \beta) + \frac{\beta}{2} = \gamma_1 - \frac{\beta}{2}.$$

Note that the parameter β of DilithiumRK is twice that of Dilithium with uncompressed public keys. Now, we need to prove that $c^\star = H(m^\star \| w_1^\circ)$, where $w_1^\circ = \mathbf{HighBits}_q(\mathbf{A}z^\circ - c^\star t, 2\gamma_2)$. To do so, it is enough to show that $w_1^\star = w_1^\circ$, i.e., $\mathbf{HighBits}_q(\mathbf{A}z^\star - c^\star t^\star, 2\gamma_2) = \mathbf{HighBits}_q(\mathbf{A}z^\circ - c^\star t, 2\gamma_2)$.

On one hand, we have:

$$\mathbf{A}z^\star - c^\star t^\star = \mathbf{A}z^\star - c^\star(t + \mathbf{A}s_1^\star + s_2^\star) = \mathbf{A}z^\star - c^\star t - c^\star \mathbf{A}s_1^\star - c^\star s_2^\star.$$

On the other hand:

$$\mathbf{A}z^\circ - c^\star t = \mathbf{A}(z^\star - c^\star s_1^\star) - c^\star t = \mathbf{A}z^\star - \mathbf{A}c^\star s_1^\star - c^\star t = \mathbf{A}z^\star - c^\star t - c^\star \mathbf{A}s_1^\star.$$

Therefore: $\mathbf{A}z^\star - c^\star t^\star = \mathbf{A}z^\circ - c^\star t - c^\star s_2^\star$. Since $\| \mathbf{LowBits}_q(\mathbf{A}z^\star - c^\star t^\star) \|_\infty \leq \gamma_2 - \beta$ (from Eq. 2) and $\| c^\star s_2^\star \|_\infty \leq \beta/2 < \beta$, by Lemma 2 we conclude that:

$$\mathbf{HighBits}_q(\mathbf{A}z^\star - c^\star t^\star, 2\gamma_2) = \mathbf{HighBits}_q(\mathbf{A}z^\circ - c^\star t, 2\gamma_2).$$

Thus, $w_1^\star = w_1^\circ$, and consequently $c^\star = H(m^\star \| w_1^\circ)$, as required. Notice that $m^\star \notin$ MList. Therefore, $(m^\star, \sigma^\circ = (c^\star, z^\circ))$ is a valid forgery for the Dilithium signature scheme. We note that the environment of $\mathcal{A}$ is perfectly simulated, and whenever $\mathcal{A}$ wins the game $\mathbf{Uf\text{-}CMA\text{-}HRK}_{\text{DilithiumRK}}$, $\mathcal{B}$ wins the game $\mathbf{Uf\text{-}CMA}_{\text{Dilithium}}$.

□

Remark 1. Note that in the above proof of Theorem 1, when the reduction adversary B responds to signature queries made by $\mathcal{A}$, the number of repetitions that $\mathcal{B}$ must perform to obtain a valid signature is quite small (approximately 5 for the Dilithium parameters [BDK+21]). This number corresponds exactly to the number of rejections encountered when signing with a private key of the form $sk = (\mathbf{A}, s_1'', s_2'') = (\mathbf{A}, s_1 + s_1', s_2 + s_2')$. As noted by the authors of Dilithium, the number of repetitions required to obtain a valid z during signing does not depend on the secret elements (see Section 3.4, "Number of Repetitions", of the Dilithium Round 3 paper [BDK+21]). Instead, it depends solely on public parameters: γ_1, γ_2, β, and the dimension (k, l) of the public matrix $\mathbf{A}$. Concretely, this expected number of repetitions is approximately $\exp(-256\beta(l/\gamma_1 + k/\gamma_2))$. In practice, this value is around 4 for Dilithium levels 2 and 5, and about 5 for Dilithium level 3 (see Table 2 of Dilithium paper [BDK+21]).

4.3 Unlinkability of DilithiumRK

To prove that our signature scheme with rerandomizable keys DilithiumRK has the unlinkability of session public keys property 1, it suffices to show the following holds.

i) The advantage $\mathrm{Adv}^{\mathrm{ind-pk},n}_{\mathrm{DilithiumRK}}(\mathcal{B})$ is negligible for any adversary $\mathcal{B}$;

ii) DilithiumRK permits signing with oracle reprogramming;

iii) The $L1$ distance $\delta_{\mathrm{DilithiumRK}}$ is negligible;

iv) The advantage of finding forgeries for DilithiumRK is negligeable.

Condition iv) is already proven in the previous section, let us now prove the 3 first conditions.

A) Proof of the independent session public keys: Since we have assumed that the matrix $\mathbf{A}$ is shared within the network, one can omit the first component of public keys in the analysis; and thus considering that a public key is in the form $t = \mathbf{A}s_1 + s_2$ instead of $(\mathbf{A}, t)$.

Lemma 3. *Let ρ be a randomness in $\mathcal{R}$. Let $(pk^0, .) \leftarrow \mathrm{DilithiumRK.KGen}(1^\lambda)$ and $(pk^1, .) \leftarrow \mathrm{DilithiumRK.KGen}(1^\lambda)$ be two public keys. If $\mathbf{D} - \mathbf{MLWE}_{\mathbf{A},\mathcal{U}}$ is hard, then an adversary $\mathcal{A}$ who observes two session public keys*

$$pk_0 = \mathrm{DilithiumRK.UpdatePk}(pk^0, \rho) \text{ and}$$
$$pk_1 = \mathrm{DilithiumRK.UpdatePk}(pk^1, \rho)$$

have a negligeable advantage to distinguish the two keys. Moreover, the advantage of distinguishing the session public keys is less or equal than $2\mathrm{Adv}^{\mathrm{D-MLWE}}_{\mathbf{A},\mathcal{U}}$

Proof. Suppose that the problem $\mathbf{D} - \mathbf{MLWE}_{\mathbf{A},\mathcal{U}}$ is hard. Let $pk^0 = t = \mathbf{A}s_1^0 + s_2^0$ and $pk^1 = t' = \mathbf{A}s_1^1 + s_2^1$ with their corresponding secret keys $sk^0 = (s_1^0, s_2^0) \in S_\eta^l \times S_\eta^k$ and $sk^1 = (s_1^1, s_2^1) \in S_\eta^l \times S_\eta^k$. Let $(s_1^\rho, s_2^\rho) \in S_\eta^l \times S_\eta^k$ sampled uniformly at random with randomness ρ (e.g., ρ can be used as a seed).

- Game 0: In this game, the adversary is given the session public key $pk_0^\rho = t + \mathbf{A}s_1^\rho + s_2^\rho$.
- Game 1: The same as Game 0, but t is now sampled randomly and uniformly in R_q^k. By the hardness of $\mathbf{D} - \mathbf{MLWE}_{\mathbf{A},\mathcal{U}}$, the adversary have a negligible advantage to distinguish between Game 0 and Game 1.
- Game 2: The same as Game 1 but we replace t by $t' = \mathbf{A}s_1^1 + s_2^1$. One cannot distinguish between Game 2 and Game 1 thanks to the hardness of $\mathbf{D} - \mathbf{MLWE}_{\mathbf{A},\mathcal{U}}$ problem.

This show an adversary cannot distinguish Game 0 (first session public key) and Game 2 (second session public key) if $\mathbf{D-MLWE}_{\mathbf{A},\mathcal{U}}$ is hard. □

Theorem 2. *The signature* DilithiumRK *has the property of independent session public keys i.e The following holds:*

$$\mathrm{Adv}^{\mathrm{ind-pk},n}_{\mathrm{DilithiumRK}}(\mathcal{A}) \leq 2n\mathrm{Adv}^{\mathrm{D-MLWE}}_{\mathbf{A},\mathcal{U}},$$

provided that the problem $\mathbf{D-MLWE}_{\mathbf{A},\mathcal{U}}$ *is hard.*

Proof. Let $pk^0, pk^1, pk^2, \ldots, pk^n$ be $(n+1)$ public keys generated independently from DilithiumRK.KGen(). Let $\rho_1, \rho_2, \ldots, \rho_n$ be n randomness. Consider the following distribution output from DilithiumRK.UpdatePk invoked upon a fixed public key (pk^0) :

$$\begin{aligned} K = [K_1, K_2, \ldots, K_n] :=& [\mathrm{DilithiumRK.UpdatePk}(pk^0, \rho_1) \\ & , \mathrm{DilithiumRK.UpdatePk}(pk^0, \rho_2) \\ & , \ldots \\ & , \mathrm{DilithiumRK.UpdatePk}(pk^0, \rho_n)] \end{aligned}$$

And the following distribution output from DilithiumRK.UpdatePk invoked upon freshly generated public keys (pk^i):

$$\begin{aligned} L = [L_1, L_2, \ldots, L_n] :=& [\mathrm{DilithiumRK.UpdatePk}(pk^1, \rho_1) \\ & , \mathrm{DilithiumRK.UpdatePk}(pk^2, \rho_2) \\ & , \ldots \\ & , \mathrm{DilithiumRK.UpdatePk}(pk^n, \rho_n)] \end{aligned}$$

Notice that both distributions use the same update ρ_i for $i = 0, 1, \ldots, n$. For $j = n, \ldots, 0$, let

$$G^{(j)} := [L_1, L_2, \ldots, L_{n-j}, K_{n-j+1}, \ldots, K_n]$$

be the distribution defined in terms of K and L. Remark that $G^{(0)} = L$, $G^{(n)} = K$ and $|G^{(j)}| = n$. Now, let us compute $|Pr[\mathcal{A}^{G^{(0)}} = 1] - Pr[\mathcal{A}^{G^{(n)}} = 1]|$. By the triangle inequality, we know that

$$\begin{aligned} |Pr[\mathcal{A}^{G^{(0)}} = 1] - Pr[\mathcal{A}^{G^{(n)}} = 1]| \leq & |Pr[\mathcal{A}^{G^{(0)}} = 1] - Pr[\mathcal{A}^{G^{(1)}} = 1]| \\ & + |Pr[\mathcal{A}^{G^{(1)}} = 1] - Pr[\mathcal{A}^{G^{(2)}} = 1]| \\ & + \ldots \\ & + |Pr[\mathcal{A}^{G^{(n-1)}} = 1] - Pr[\mathcal{A}^{G^{(n)}} = 1]| \end{aligned}$$

The difference between $G^{(j)}$ and $G^{(j+1)}$ for $j = 0, \ldots, n-1$ simplifies to a difference between $K_{n-j} = t + As_1 + s_2$ and $L_{n-j} = t + As_1' + s_2'$ for t a uniform element of R_q^k. By Lemma 3, the advantage of such samples is $\leq$ two times the advantage of solving the $\mathbf{D-MLWE}_{\mathbf{A},\mathcal{U}}$. Therefore

$$|Pr[\mathcal{A}^{G^{(0)}} = 1] - Pr[\mathcal{A}^{G^{(n)}} = 1]| = \mathrm{Adv}^{\mathrm{ind-pk},n}_{\mathrm{DilithiumRK}}(\mathcal{A}) \leq 2n\mathrm{Adv}^{\mathrm{D-MLWE}}_{\mathbf{A},\mathcal{U}}$$

□

B) Proof of naHVZK: Let us first define DilithiumRK-IDS : the identification scheme that can be transformed into a signature scheme to get DilithiumRK.

$\underline{\text{Prv}_1(sk = (\mathbf{A}, s_1, s_2)) :}$
1: $y \leftarrow \mathcal{U}(S^l_{\gamma_1 - 1})$;
2: $w := \mathbf{A}y$;
3: $w_1 := \mathbf{HighBits}_q(w, 2\gamma_2)$;
4: **return** $(W = w_1, St = (w, y))$

$\underline{\text{Vrf}_1(pk = (\mathbf{A}, t), W = w_1) :}$
1: $c \leftarrow \mathcal{U}(B_\tau)$;
2: **return** c

$\underline{\text{IGen}() :}$
1: $(s_1, s_2) \leftarrow \mathcal{U}(S^l_\eta) \times \mathcal{U}(S^k_\eta)$;
2: $t := \mathbf{A}s_1 + s_2$;
3: $pk := (\mathbf{A}, t), sk := (\mathbf{A}, s_1, s_2)$
4: **return** (pk, sk)

$\underline{\text{Prv}_2(sk = (\mathbf{A}, s_1, s_2), c, St = (w, y)) :}$
1: $z := y + cs_1$;
2: **if** $\| z \|_\infty \geq \gamma_1 - \beta$ **then**
3: **return** $\perp$
4: **end if**
5: **if** $\| \mathbf{LowBits}_q(w - cs_2, 2\gamma_2) \|_\infty \geq \gamma_2 - \beta$ **then**
6: **return** $\perp$
7: **end if**
8: **return** $Z = z$;

$\underline{\text{Vrf}_2(pk = (\mathbf{A}, t_1), W = w_1, c, Z = (z, h)) :}$
1: $b_1 := [\| z \|_\infty < \gamma_1 - \beta]$;
2: $b_2 := [w_1 = \mathbf{HighBits}_q(h, \mathbf{A}z - ct, 2\gamma_2)]$;
3: **return** $b_1 \wedge b_2$

Figure 13: DilithiumRK-IDS : the identification scheme that can be transformed into a signature scheme to get DilithiumRK.

It is easy to show that DilithiumRK $= FS_\infty[\text{DilithiumRK} - \text{IDS}, H]$. Let us prove now that DilithiumRK-IDS is perfectly naHVZK. We define the following transcripts.

$\underline{\text{Trans}(sk = (\mathbf{A}, s_1, s_2)) :}$
1: $y \leftarrow \mathcal{U}(S^l_{\gamma_1 - 1})$;
2: $w := \mathbf{A}y$;
3: $c \leftarrow B_\tau$;
4: $z := y + cs_1$;
5: **if** $\| \mathbf{LowBits}_q(w - cs_2, 2\gamma_2) \|_\infty \geq \gamma_2 - \beta$ **then**
6: **return** $\perp$;
7: **end if**
8: **if** $\| z \|_\infty \geq \gamma_1 - \beta$ **then**
9: **return** $\perp$;
10: **end if**
11: **return** (c, z)

$\underline{\text{Sim}(pk = (\mathbf{A}, t)) :}$
1: **return** $\perp$ with probability $1 - \dfrac{\left|S^l_{\gamma_1 - \beta - 1}\right|}{\left|S^l_{\gamma_1 - 1}\right|}$;
2: $c \leftarrow \mathcal{U}(B_\tau)$;
3: $z \leftarrow \mathcal{U}\left(S^l_{\gamma_1 - \beta - 1}\right)$;
4: **if** $\| \mathbf{LowBits}_q(Az - ct, 2\gamma_2) \|_\infty \geq \gamma_2 - \beta$ **then**
5: **return** $\perp$
6: **end if**
7: **return** (c, z)

Figure 14: Left: A genuine transcript generated by the Trans(sk) algorithm using a secret key sk; Right: A simulated transcript produced by the Sim(pk) algorithm using a public key pk.

Lemma 4. *If* $\beta \geq \max_{s \in S_{2\eta}, c \in B_\tau} \| cs \|_\infty$ *then DilithiumRK-IDS is perfectly naHVZK (with* $\epsilon_{zk} = 0$*).*

Proof. The proof of this lemma can be derived from the proof of perfect naHVZK property of Dilithium-QROM-IDS as established in [KLS18]. However, for the sake of completeness, let us provide the proof here. The objective is twofold: first, to demonstrate that $\perp$ is equally likely to be returned by both Trans(sk) and Sim(pk), and second, to show that the distribution of (c, z) is the same for both algorithms.

Let us compute the probability that a given $z \in S^l_{\gamma_1 - \beta - 1}$ is generated in Line 4 of Trans(sk) when $sk = (\mathbf{A}, s_1, s_2)$ with $s_1 = s'_1 + s''_1 \in S^l_{2\eta}$ and $s_2 = s'_2 + s''_2 \in S^k_{2\eta}$. For any $c \in B_\tau$, we have

$$\Pr[y + cs_1 = z | y \leftarrow \mathcal{U}(S^l_{\gamma_1 - 1})] = \Pr[y + c(s'_1 + s''_1) = z | y \leftarrow \mathcal{U}(S^l_{\gamma_1 - 1})]$$
$$= \Pr[y = z - c(s'_1 + s''_1) | y \leftarrow \mathcal{U}(S^l_{\gamma_1 - 1})]$$

Notice that $z - c(s'_1 + s''_1)$ belongs to the set $S^l_{\gamma_1 - 1}$ because the infinity norm of $c(s'_1 + s''_1)$ is bounded by β. This implies that the probability $\Pr[y + cs_1 = z | y \leftarrow \mathcal{U}(S^l_{\gamma_1 - 1})]$ is equal to $1/\left|S^l_{\gamma_1 - 1}\right|$.

Therefore, every element z in the set $S^l_{\gamma_1 - \beta - 1}$ has an equal probability of being generated. Furthermore, the probability of producing a value z in $S^l_{\gamma_1 - \beta - 1}$ and, consequently, not returning $\perp$ in Line 9 of Trans(sk) is precisely $\left|S^l_{\gamma_1 - \beta - 1}\right| / \left|S^l_{\gamma_1 - 1}\right|$. Thus, after the completion of Line 9, either $\perp$ has been returned (with probability $1 - \left|S^l_{\gamma_1 - \beta - 1}\right| / \left|S^l_{\gamma_1 - 1}\right|$), or the distribution of z is uniform within $S^l_{\gamma_1 - \beta - 1}$.

□

Corollary 1. *The L1 distance* $\delta_{\text{DilithiumRK}} = 0$.

Proof. It follows from the fact that DilithiumRK is perfectly naHVZK by the previous Lemma. □

C) Proof of signing with oracle reprogramming: Let us now show that our IDS scheme permits signing with oracle reprogramming. Let us define Forge and Ext functions in the following figure.

$\text{Forge}_{DRK}(m, pk = (\mathbf{A}, t))$:	$\text{Ext}_{DRK}(m, \sigma = (c, z), pk = (\mathbf{A}, t))$:
1: $(c, z) := \text{Sim}(pk = (\mathbf{A}, t))$;	1: $w_1 := \mathbf{HighBits}_q(\mathbf{A}z - ct, 2\gamma_2)$;
2: **return** $(y = c, \sigma = (c, z))$	2: **return** $(m \parallel w_1)$

Figure 15: Signing with oracle reprogramming

Lemma 5. DilithiumRK *permits signing with oracle reprogramming.*

Proof. Let $(c, \sigma = (c, z)) := \text{Forge}_{DRK}(m, pk = (\mathbf{A}, t))$. According to Figure 15, we can observe that $\text{Ext}_{DRK}(m, \sigma = (c, z), pk = (\mathbf{A}, t)) = (m \parallel w_1)$, where $w_1 = \mathbf{HighBits}_q(\mathbf{A}z - ct, 2\gamma_2)$. Consequently, we have $c = H(\text{Ext}_{DRK}(m, \sigma, pk)) \Leftrightarrow H(m \parallel w_1) = c$. Furthermore, DilithiumRK.Verify$(m, \sigma = (c, z), pk = (\mathbf{A}, t)) = 1$ since, by the definition of the Forge_{DRK} function, we have $\parallel z \parallel_\infty < \gamma_1 - \beta$. Thus,

$$\text{DilithiumRK.Verify}^{H:\text{Ext}_{DRK}(m,\sigma,pk) \mapsto c}(m, \sigma, pk) = 1$$

This show that DilithiumRK permits signing with oracle reprogramming. □

We have the following theorem for the unlinkability property.

Theorem 3. *For any adversary* $\mathcal{A}$ *that makes* q_H *random oracle queries and* n *update queries, there exists an algorithm* $\mathcal{B}$ *such that*

$$\text{Adv}^{unl-pk}_{\mathcal{A},\text{DilithiumRK}} \leq 2n\text{Adv}^{\text{MLWE}}_{\mathbf{A},\mathcal{U}} + q_H \text{Adv}^{uf-cma-hrk}_{\text{DilithiumRK}}(\mathcal{B})$$

Proof. It follows from Lemma 1 and previous results. □

5 On the rerandomizability of keys for Dilithium with public key compression

In this section, we show how one can design a signature scheme with rerandomizable keys from Dilithium with public key compression. Notice that $\mathbf{Power2Round}_q$ function is not homomorphic i.e.

$$\text{Let } \mathrm{P2R}(r,d) = r_1 \text{ where } (r_1, r_0) := \mathbf{Power2Round}_q(r,d).$$

Then $\mathrm{P2R}(t,d) + \mathrm{P2R}(t',d)$ is not always equal to $\mathrm{P2R}(t+t',d)$. We only have $\mathrm{P2R}(t+t',d) = \mathrm{P2R}(t,d) + \mathrm{P2R}(t',d) + t_{error}$ when $t_{error} \in R_q^k$ is an error term. So a possible solution could be to add the error as a hint that will be included in the signature so that when we have a signature $\sigma = (c, z, h, t_{error})$, the verifier with a session public key $(\rho, t'' = \mathrm{P2R}(t,d) + \mathrm{P2R}(t',d))$ using the error t_{error} can retrieve the correct session public key. In order to have an efficient signature, we need the number of encoded bytes for t_{error} to be very small. Then we need the encoding of t_{error} coefficients to be very small. We argue that this is the case for Dilithium and make the following conjecture.

Conjecture. Let $a, b \in \mathbb{Z}/q\mathbb{Z}$ and $c = a + b$. Let $a_1 = \mathrm{P2R}(a,d), b_1 = \mathrm{P2R}(b,d)$, $c_1 = \mathrm{P2R}(c,d)$ and $c_1' = \mathrm{P2R}(2^d(a_1+b_1),d)$. Then $c_1 = c_1' + e$ for some $e \in \{-1, 0, 1, e_4, e_5, e_6, e_7\}$ mod q i.e. e can take at most 7 distinct values in $\mathbb{Z}/q\mathbb{Z}$.

Notice that e can be encoded with 3 bits. For example for Dilithium parameters ($q = 8380417$ and $d = 13$), we found $e \in \{-1, 0, 1, -1022, -1023\} \mod q$. Note that $-1023 = -2^{d-3} + 1 \equiv 48379394 \mod q$ and $-1022 = -2^{d-3} + 2 \equiv 8379395 \mod q$. Using this result, we design an efficient signature with rerandomizable keys from Dilithium with compressed public keys, called DilithiumRKC, as follows.

DilithiumRKC.KGen() :

1: $\zeta \leftarrow \{0,1\}^{256}$;
2: $(_, \rho', K) := H(\zeta)$;
3: $(s_1, s_2) \in S_\eta^l \times S_\eta^k := \mathbf{ExpandS}(\rho')$;
4: $\rho := \mathbf{getSeedRhoFromDB}()$;
5: $\mathbf{A} \sim R_q^{k \times l} := \mathbf{ExpandA}(\rho)$;
6: $t := \mathbf{A}s_1 + s_2$;
7: $(t_1, t_0) := \mathbf{Power2Round}_q(t,d)$;
8: $tr \in \{0,1\}^{384} := \mathbf{CRH}(\rho \parallel t_1)$;
9: $t_1^{error} := 0 \in R_q^k$;
10: $pk := (\rho, t_1)$;
11: $sk := (\rho, K, tr, s_1, s_2, t_0, t_1^{error})$;
12: **return** (pk, sk)

DilithiumRKC.Sign(m, sk) :

1: Parse $(\rho, K, tr, s_1, s_2, t_0, t_1^{error}) := sk$;
2: $sk2 := (\rho, K, tr, s_1, s_2, t_0)$;
3: (c, (z, h)) := Dilithium.Sign$(m, sk2)$);
4: $h := (h, t_1^{error})$;
5: **return** $\sigma = (c, (z, h))$

DilithiumRKC.Verify(m, σ, pk) :

1: $(\rho, t_1) := pk$;
2: Parse $(c, (z, (h, t_1^{error}))) := \sigma$
3: $\sigma_2 := (c, (z, h))$;
4: $t_{12} := t_1 + t_1^{error}$;
5: $b :=$ Dilithium.Verify$(m, \sigma_2, (\rho, t_{12}))$;
6: **return** b

Figure 16: Key generation, signing and verification algorithms of DilithiumRKC

DilithiumRKC.UpdateSk(sk, μ) :

$(\rho, K, tr, s_1, s_2, t_0, t_1^{error}) := sk$;
$\mathbf{A} \sim R_q^{k\times l} := \mathbf{ExpandA}(\rho)$;
$t_1 := ((\mathbf{A}s_1 + s_2) - t_0)/2^d$;
$(s_1', s_2') := \text{ExpandSP}(\rho \parallel t_1 \parallel \mu)$;
$s_1'' := s_1 + s_1'$; $s_2'' := s_2 + s_2'$;
$t'' := \mathbf{A}s_1'' + s_2''$;
$(t_1'', t_0'') := \mathbf{Power2Round}_q(t'', d)$;
$tr'' \in \{0,1\}^{384} := \mathbf{CRH}(\rho \parallel t_1')$;
$t' := \mathbf{A}s_1' + s_2'$; $t_1' := \text{P2R}(t', d)$;
$\widehat{t_1}' := \text{P2R}(2^d(t_1 + t_1'), d)$; $t_{12}^{error} := t_1'' - \widehat{t_1}'$;
$sk' := (\rho, K, tr'', s_1'', s_2'', t_0'', t_{12}^{error})$;
return sk'

DilithiumRKC.UpdatePk(pk, μ) :

$(\rho, t_1) := pk$;
$(s_1', s_2') := \text{ExpandSP}(\rho \parallel t_1 \parallel \mu)$;
$\mathbf{A} \sim R_q^{k\times l} := \mathbf{ExpandA}(\rho)$;
$t'' := \mathbf{A}s_1' + s_2'$;
$t_1'' := \text{P2R}(t'', d)$;
$t_1' := \text{P2R}(2^d(t_1 + t_1''), d)$;
$pk' := (\rho, t_1')$;
return pk'

Figure 17: Public and private keys update algorithms of DilithiumRKC

Notice that we only add $3 \times 256 \times \frac{k}{8} = 96k$ bytes to the Dilithium signature size when encoding each coefficient of the elements in the vector t_1^{error} with 3 bits. On the other hand, when we include t_0 in the public key, as in DilithiumRK (thus considering the whole t), the public key size increases by $416k$ bytes. When we consider Dilithium and we include t_0 in the public key (e.g. $pk := (\rho, t_1, t_0)$), we call DilithiumRK0, the corresponding signature scheme with rerandomizable keys. We introduce its different algorithms in the following figures (Fig. 18 and Fig. 19).

DilithiumRK0.KGen() :

$\zeta \leftarrow \{0,1\}^{256}$; $(_, \rho', K) := H(\zeta)$;
$(s_1, s_2) \in S_\eta^l \times S_\eta^k := \mathbf{ExpandS}(\rho')$;
$\rho := \mathbf{getSeedRhoFromDB}()$;
$A \sim R_q^{k\times l} := \mathbf{ExpandA}(\rho)$;
$t := As_1 + s_2$;
$(t_1, t_0) := \mathbf{Power2Round}_q(t, d)$;
$tr \in \{0,1\}^{384} := \mathbf{CRH}(\rho \parallel t_1)$;
$pk := (\rho, t_1, t_0)$;
$sk := (\rho, K, tr, s_1, s_2, t_0)$;
return (pk, sk)

DilithiumRK0.Sign(m, sk) :

$(c, (z, h)) := \text{Dilithium.Sign}(m, sk)$;
return $\sigma = (c, (z, h))$

DilithiumRK0.Verify(m, σ, pk) :

$(\rho, t_1, t_0) := pk$;
$b := \text{Dilithium.Verify}(m, \sigma, pk = (\rho, t_1))$;
return b

Figure 18: Key generation, signing and verification algorithms of DilithiumRK0

DilithiumRK0.UpdatePk(pk, μ) :

$(\rho, t_1, t_0) := pk$;
$(s_1', s_2') := \text{ExpandSP}(\rho \parallel t_0 \parallel \mu)$;
$A \sim R_q^{k\times l} := \mathbf{ExpandA}(\rho)$;
$t := t_1 2^d + t_0$:
$t' := As_1' + s_2'$;
$t'' := t + t'$
$(t_1'', t_0'') := \mathbf{Power2Round}_q(t'', d)$;
$pk' := (\rho, t_1'', t_0'')$;
return pk'

DilithiumRK0.UpdateSk(sk, μ) :

$(\rho, K, tr, s_1, s_2, t_0) := sk$;
$(s_1', s_2') := \text{ExpandSP}(\rho \parallel t_0 \parallel \mu)$;
$s_1'' := s_1 + s_1'$; $s_2'' := s_2 + s_2'$;
$A \sim R_q^{k\times l} := \mathbf{ExpandA}(\rho)$;
$t'' := As_1'' + s_2''$;
$(t_1'', t_0'') := \mathbf{Power2Round}_q(t'', d)$;
$tr'' \in \{0,1\}^{384} := \mathbf{CRH}(\rho \parallel t_1'')$;
$sk' := (\rho, K, tr'', s_1'', s_2'', t_0'')$;
return sk'

Figure 19: Public and private keys update algorithms of DilithiumRK0

Notice the difference between the key generation algorithms of Dilithium and DilithiumRK0 is that we have added the lowbits t_0 of t in the public key of DilithiumRK0.

6 Implementation

We have provided a proof-of-concept implementation[1] in Python for our principal signature scheme with rerandomizable keys DilithiumRK, as well as for its variants, DilithiumRK0 and DilithiumRKC. We compare these schemes (for security level $\geq$ 128 bits) with Dilithium version 3.1 from NIST Round 3, and the parameters are summarized in the following table 1.

Table 1: Different parameters for DilithiumRK, DilithiumRK0, DilithiumRKC, and Dilithium levels 2 and 3.

Parameters	Dilithium 2	Dilithium 3	DilithiumRK	DilithiumRK0	DilithiumRKC
q	8380417	8380417	8380417	8380417	8380417
n	256	256	256	256	256
(k,l)	$(4,4)$	$(6,5)$	$(5,5)$	$(5,5)$	$(5,5)$
d	13	13	13	13	13
τ	39	49	50	50	50
η	2	4	2	2	2
β	78	196	200	200	200
γ_1	2^{17}	2^{19}	2^{19}	2^{19}	2^{19}
γ_2	$(q-1)/88$	$(q-1)/32$	$(q-1)/32$	$(q-1)/32$	$(q-1)/32$
$\|pk\|$	1312	1952	3712	3712	1632
$\|\sigma\|$	2420	3293	3232	3317	3797
$\|pk\|+\|\sigma\|$	3732	5245	6944	7029	5429

Remark 2. Notice that for the computation cost, using our non-optimized implementation with Python, DilithiumRK signing takes about 25 ms and verification about 7 ms. In comparison, Dilithium at security level 2 requires only about 9 ms for signing and 4 ms for verification. Our experimental environment is Ubuntu 24.04.1 LTS (GNU/Linux 5.15.153.1-microsoft-standard-WSL2 x86_64) on Intel(R) Core(TM) i7-150U 1.8 GHz with 16 GB RAM.

Impact on Bitcoin Network. In Bitcoin-like cryptocurrencies, signature schemes are used to authenticate transactions which are grouped into blocks in the ledger. A Bitcoin transaction [Dev] contains the sender's public key pk_s, its signature σ_s, and additional data. The combined size $|pk| + |\sigma|$ affects transaction throughput. Bitcoin is designed so that a block is created approximately every 10 minutes, and the block size is also limited ($\leq$ 4 million weight units). Given these two constraints, minimizing this combined size is crucial for improving transaction throughput. In what follows, we compare our scheme with other lattice-based deterministic wallets. Since available data for other schemes corresponds to a security level of 160 bits (see Table 2 [HLZ24]), we propose parameters $((k,l) = (6,6), \tau = 60, \eta = 2, \beta = 240)$ for our schemes to achieve this level of security.

Table 2: Our signature schemes compared to other lattice-based signature schemes with rerandomizable keys for a security level of 160 bits.

Scheme	Base Signature	$\|pk\|$ in byte	$\|\sigma\|$ in byte	$\|pk\|+\|\sigma\|$ in byte
Alkadri et al. [AADE+20]	qTESLA	36549	5610	42159
Hu et al. [HLZ24]	Falcon	897	23755	24652
DilithiumRK (ours)	Dilithium	4448	3872	**8320**
DilithiumRKC (ours)	Dilithium	1952	4529	**6481**

[1] https://github.com/mseckept/dilithium-rk-wallet.git

7 Conclusion

In this paper, we demonstrated that the Dilithium signature scheme can be used to construct efficient deterministic wallets with provable security, which appear to be more suitable for post-quantum Bitcoin compared to other lattice-based deterministic wallets. We have constructed DilithiumRK a signature scheme with rerandomizable keys and proved the unforgeability and unlinkabilty properties. The public keys of our primary signature scheme, DilithiumRK, are uncompressed. To reduce the size of the public keys, we introduced DilithiumRKC, our most compact proposal. This scheme incorporates a new hint in the signature, and it would be interesting to analyze its impact on the unlinkability property in future research.

Acknowledgements

The authors thank Julien Devevey for valuable comments and suggestions and Corentin Jeudy for helpful discussions. They would also like to thank the anonymous reviewers for their valuable feedback, which helped improve the quality of this paper.

References

[AADE+20] Nabil Alkeilani Alkadri, Poulami Das, Andreas Erwig, Sebastian Faust, Juliane Krämer, Siavash Riahi, and Patrick Struck. Deterministic wallets in a quantum world. In *Proceedings of the 2020 ACM SIGSAC Conference On Computer And Communications Security*, pages 1017–1031, 2020. doi:10.1145/3372297.3423361.

[ABB+20] Erdem Alkim, Paulo SLM Barreto, Nina Bindel, Juliane Krämer, Patrick Longa, and Jefferson E Ricardini. The lattice-based digital signature scheme qtesla. In *International Conference on Applied Cryptography and Network Security*, pages 441–460. Springer, 2020. doi:10.1007/978-3-030-57808-4_22.

[BBD+23] Manuel Barbosa, Gilles Barthe, Christian Doczkal, Jelle Don, Serge Fehr, Benjamin Grégoire, Yu-Hsuan Huang, Andreas Hülsing, Yi Lee, and Xiaodi Wu. Fixing and mechanizing the security proof of fiat-shamir with aborts and dilithium. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023*, volume 14083 of *Lecture Notes in Computer Science*, pages 358–389, Cham, 2023. Springer Nature Switzerland. doi:10.1007/978-3-031-38554-4_12.

[BDK+21] Shi Bai, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS–dilithium: Algorithm specification and supporting documentation (version 3.1). Round-3 submission to the NIST Post-Quantum Cryptography Standardization Project, 2021. URL: https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf.

[BJRW23] Katharina Boudgoust, Corentin Jeudy, Adeline Roux-Langlois, and Weiqiang Wen. On the hardness of module learning with errors with short distributions. *J. Cryptol.*, 36(1):1, 2023. doi:10.1007/s00145-022-09441-3.

[DEF+21] Poulami Das, Andreas Erwig, Sebastian Faust, Julian Loss, and Siavash Riahi. The exact security of bip32 wallets. In *Proceedings of the 2021 ACM SIGSAC*

conference on computer and communications security, pages 1020–1042, 2021. doi:10.1145/3460120.3484807.

[DEF+25] Poulami Das, Andreas Erwig, Sebastian Faust, Philipp-Florens Lehwalder, Julian Loss, Ziyan Qu, and Siavash Riahi. Bip32-compatible threshold wallets. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security (AsiaCCS '25)*, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3708821.3710830.

[DEMS24] Poulami Das, Andreas Erwig, Michael Meyer, and Patrick Struck. Efficient post-quantum secure deterministic threshold wallets from isogenies. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, pages 522–532, 2024. doi:10.1145/3634737.3657008.

[Dev] Bitcoin Develepers. Bitcoin transactions. URL: https://developer.bitcoin.org/reference/transactions.html.

[DFL19] Poulami Das, Sebastian Faust, and Julian Loss. A formal treatment of deterministic wallets. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 651–668, 2019. doi:10.1145/3319535.3354236.

[DFPS23] Julien Devevey, Pouria Fallahpour, Alain Passelègue, and Damien Stehlé. A detailed analysis of fiat-shamir with aborts. In *Annual International Cryptology Conference*, pages 327–357. Springer, 2023. doi:10.1007/978-3-031-38554-4_11.

[DKL+18] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 238–268, 2018. doi:10.13154/tches.v2018.i1.238-268.

[ELW23] Edward Eaton, Tancrède Lepoint, and Christopher A. Wood. Security analysis of signature schemes with key blinding. Cryptology ePrint Archive, Paper 2023/380, 2023. URL: https://eprint.iacr.org/2023/380.

[ER22] Andreas Erwig and Siavash Riahi. Deterministic wallets for adaptor signatures. In *European Symposium on Research in Computer Security*, pages 487–506. Springer, 2022. doi:10.1007/978-3-031-17146-8_24.

[ESS21] Edward Eaton, Douglas Stebila, and Roy Stracovsky. Post-quantum key-blinding for authentication in anonymity networks. In *Progress in Cryptology–LATINCRYPT 2021: 7th International Conference on Cryptology and Information Security in Latin America, Bogotá, Colombia, October 6–8, 2021, Proceedings 7*, pages 67–87. Springer, 2021. doi:10.1007/978-3-030-88238-9_4.

[FHK+18] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, Zhenfei Zhang, et al. Falcon: Fast-fourier lattice-based compact signatures over ntru. Technical Report 36(5), NIST Post-Quantum Cryptography Standardization Project, 2018. Submission to the NIST's Post-Quantum Cryptography Standardization Process.

[FKM+16] Nils Fleischhacker, Johannes Krupp, Giulio Malavolta, Jonas Schneider, Dominique Schröder, and Mark Simkin. Efficient unlinkable sanitizable signatures from signatures with re-randomizable keys. In *Public-Key Cryptography–PKC 2016: 19th IACR International Conference on Practice and Theory in Public-Key Cryptography, Taipei, Taiwan, March 6-9, 2016, Proceedings, Part I*, pages 301–330. Springer, 2016. doi:10.1007/978-3-662-49384-7_12.

[GS15] Gus Gutoski and Douglas Stebila. Hierarchical deterministic bitcoin wallets that tolerate key leakage. In *International Conference on Financial Cryptography and Data Security*, pages 497–504. Springer, 2015. doi:10.1007/978-3-662-47854-7_31.

[GS22] Jens Groth and Victor Shoup. On the security of ecdsa with additive key derivation and presignatures. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 365–396. Springer, 2022. doi:10.1007/978-3-031-06944-4_13.

[HLZ24] Mingxing Hu, Zhen Liu, and Yunhong Zhou. Efficient post-quantum secure deterministic wallet scheme. *Cybersecurity*, 7(1):24, 2024. doi:10.1186/s42400-024-00216-w.

[Hop13] Nicholas Hopper. Proving security of tor's hidden service identity blinding protocol. Tech report 2013-12-001, Tor Project, 2013. URL: https://www-users.cse.umn.edu/~hoppernj/basic-proof.pdf.

[Hu23] Mingxing Hu. Post-quantum secure deterministic wallet: Stateless, hot/cold setting, and more secure. Cryptology ePrint Archive, Paper 2023/062, 2023. URL: https://eprint.iacr.org/2023/062.

[KL18] Merve Can Kus Khalilov and Albert Levi. A survey on anonymity and privacy in bitcoin-like digital cash systems. *IEEE Communications Surveys & Tutorials*, 20(3):2543–2585, 2018. doi:10.1109/COMST.2018.2818623.

[KLS18] Eike Kiltz, Vadim Lyubashevsky, and Christian Schaffner. A concrete treatment of fiat-shamir signatures in the quantum random-oracle model. In *Advances in Cryptology–EUROCRYPT 2018: 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29-May 3, 2018 Proceedings, Part III 37*, pages 552–586. Springer, 2018. doi:10.1007/978-3-319-78372-7_18.

[LS15] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Des. Codes Cryptogr.*, 75(3):565–599, 2015. doi:10.1007/s10623-014-9938-4.

[LYW+21] Zhen Liu, Guomin Yang, Duncan S Wong, Khoa Nguyen, Huaxiong Wang, Xiaorong Ke, and Yining Liu. Secure deterministic wallet and stealth address: Key-insulated and privacy-preserving signature scheme with publicly derived public key. *IEEE Transactions on Dependable and Secure Computing*, 19(5):2934–2951, 2021. doi:10.1109/TDSC.2021.3078463.

[Nak08] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. *Decentralized business review*, 2008. doi:10.2139/ssrn.3440802.

[Nat13] National Institute of Standards and Technology (NIST). Digital signature standard (dss), federal information processing standard publication 186-4. Fips publication 186-4, NIST, 2013. doi:10.6028/NIST.FIPS.186-4.

[Nat24] National Institute of Standards and Technology (NIST). Module-Lattice-Based Digital Signature Standard. Federal Information Processing Standards Publication 204, U.S. Department of Commerce, August 2024. URL: https://csrc.nist.gov/pubs/fips/204/final.

[SD23] Surbhi Shaw and Ratna Dutta. Compact stateful deterministic wallet from isogeny-based signature featuring uniquely rerandomizable public keys. In *International Conference on Cryptology and Network Security*, pages 392–413. Springer, 2023. doi:10.1007/978-981-99-7563-1_18.

[SD25] Surbhi Shaw and Ratna Dutta. Post-quantum secure compact deterministic wallets from isogeny-based signatures with rerandomized keys. *Theoretical Computer Science*, 1035:115127, 2025. doi:10.1016/j.tcs.2025.115127.

[Tor] Tor. Tor directory protocol, version 3. URL: https://gitweb.torproject.org/torspec.git/tree/rend-spec-v3.txt.

[Wik18] Bitcoin Wiki. Bip32 proposal, 2018. URL: https://en.bitcoin.it/wiki/BIP_0032.

[YLY+22] Xin Yin, Zhen Liu, Guomin Yang, Guoxing Chen, and Haojin Zhu. Secure hierarchical deterministic wallet supporting stealth address. In *European Symposium on Research in Computer Security*, pages 89–109. Springer, 2022. doi:10.1007/978-3-031-17140-6_5.